Corporación Unificada Nacional de Educación Superior – CUN
Ingeniería de Sistemas

Arquitectura de Software (INGS0763)

Diseño de la arquitectura de un sistema de gestión académica mediante Google AI Studio

Juan Pablo Vargas Sánchez
Docente: Alexander Calderón Martínez

8 de noviembre de 2026

Video: https://youtu.be/XETzIcJ7SuE
Repositorio: https://github.com/juanvs1985/sga-arquitectura

Contenido

1. Introducción y descripción del problema 3
2. Objetivos 4
2.1 Objetivo general 4
2.2 Objetivos específicos 4
3. Alcance, actores, supuestos y restricciones 5
3.1 Alcance 5
3.2 Actores 5
3.3 Supuestos 5
3.4 Restricciones 5
4. Requerimientos funcionales y no funcionales 6
4.1 Requerimientos funcionales 6
4.2 Requerimientos no funcionales 6
4.3 Reglas de negocio 6
4.4 Análisis 6
5. Arquitectura seleccionada y justificación 8
5.1 Opciones que comparé 8
5.2 Por qué esta y no microservicios 8
6. Modelo del sistema 9
6.1 Casos de uso 9
6.2 Componentes (vista lógica) 11
6.3 Despliegue 12
6.4 Modelo de datos 13
6.5 Secuencia de la matrícula 16
7. Matriz de trazabilidad 19
8. Decisiones arquitectónicas y análisis de riesgos 20
8.1 Decisiones 20
8.2 Atributos de calidad 20
8.3 Riesgos y puntos únicos de falla 20
8.4 Límites y validación 20
9. Uso de Google AI Studio 21
9.1 Cómo lo usé 21
9.2 Qué hice con las respuestas 21
9.3 Prototipo en AI Studio Build 22
9.4 Lo que aprendí usando la IA 24
10. Conclusiones 25
Referencias 26
Anexo A. Evaluación de cada respuesta de la IA 27
Anexo B. Capturas de la conversación en AI Studio 31
Anexo C. Texto de los prompts 37

1. Introducción y descripción del problema

La institución de educación superior del caso maneja la información de estudiantes, docentes, programas, asignaturas, grupos, matrículas, notas y horarios en archivos y aplicaciones que no están conectados entre sí. Eso trae tres problemas: hay datos duplicados que no coinciden, las consultas son lentas porque toca buscar en varios lados, y no queda registro de quién cambió una nota, una matrícula o un rol.

En este trabajo propongo la arquitectura de software de un sistema de gestión académica (SGA) que reúna toda esa información en un solo lugar, se use desde el navegador y controle el acceso según el rol de cada usuario. Parto de los 12 requerimientos funcionales y 12 no funcionales del enunciado y llego a una arquitectura, cinco decisiones documentadas y seis modelos del sistema.

Para el diseño usé Google AI Studio (Gemini 3.8 Flash) como apoyo en siete pasos, y luego AI Studio Build para generar un prototipo navegable. No acepté todo lo que propuso la IA: en la sección 9 explico qué tomé, qué cambié y qué descarté, y por qué.

Es importante aclarar que, como dice el enunciado, no se implementó el sistema. Todo lo que describo aquí es diseño propuesto; el prototipo de Build es una maqueta con datos simulados y no la implementación de esta arquitectura.

2. Objetivos

2.1 Objetivo general

Diseñar y justificar la arquitectura de software de un sistema de gestión académica seguro, mantenible, escalable y accesible desde la web, a partir de sus requerimientos, usando Google AI Studio de forma crítica.

2.2 Objetivos específicos

1. Analizar el caso y escribir los requerimientos con criterios que se puedan verificar, junto con actores, supuestos, restricciones y reglas de negocio.
2. Escoger un estilo arquitectónico y justificarlo frente a otras opciones.
3. Representar el sistema con los modelos de casos de uso, componentes, despliegue, datos y secuencia.
4. Relacionar cada requerimiento con los componentes que lo cumplen y con la forma de comprobarlo.
5. Identificar riesgos y puntos únicos de falla, y proponer cómo validar las decisiones más críticas.
6. Documentar cómo usé AI Studio y qué hice con cada una de sus respuestas.

3. Alcance, actores, supuestos y restricciones

3.1 Alcance

El trabajo cubre los 24 requerimientos del enunciado, el diseño de la arquitectura con sus modelos, las decisiones, los riesgos y la evaluación del uso de la IA. Quedan por fuera la implementación, el diseño detallado de las pantallas, la integración real con otros sistemas de la institución (solo dejo la API preparada), y cosas que ningún requerimiento pide, como prerrequisitos de asignaturas o notificaciones por correo.

3.2 Actores

- **Administrador:** maneja usuarios, roles y permisos; registra y desactiva estudiantes y docentes; consulta la auditoría.
- **Coordinador académico:** maneja programas, planes, asignaturas, periodos y grupos; asigna docentes, matricula estudiantes y saca reportes.
- **Docente:** consulta sus grupos, registra y publica notas.
- **Estudiante:** consulta sus asignaturas, su horario y sus notas publicadas.

Como actores secundarios están el servicio de copias de seguridad y, más adelante, otras plataformas de la institución que podrían usar la API.

3.3 Supuestos

Como el caso no dice todo, tuve que asumir algunas cosas:

1. La matrícula la registra el coordinador. Dejar que el estudiante se matricule solo queda como una mejora futura.
2. "Desactivar" un estudiante o docente significa cambiarle el estado, no borrarlo, para no perder el historial.
3. Las notas tienen dos estados, borrador y publicada; el estudiante solo ve las publicadas.
4. Un usuario puede tener más de un rol (por ejemplo, un docente que también coordina) y escoge con cuál trabaja al entrar.
5. El escenario de referencia es de 300 usuarios al mismo tiempo, con picos en semanas de matrícula y cierre de notas.
6. El presupuesto de la institución es moderado.

3.4 Restricciones

La aplicación tiene que ser web y funcionar en navegadores recientes, en computador y celular. Todo el tráfico va por HTTPS, las contraseñas se guardan con hash y nunca en texto plano, y las interfaces se exponen como API REST documentada. La disponibilidad mensual debe ser de al menos 99,5 %, lo que deja como máximo unas 3,6 horas de caída no programada al mes. Y por el enunciado, no se deben usar datos personales reales en prompts, capturas ni en el repositorio.

4. Requerimientos funcionales y no funcionales

Para cada requerimiento escribí un criterio de aceptación, es decir, cómo sabría uno que se cumple. Esto me sirvió después para revisar lo que proponía la IA.

4.1 Requerimientos funcionales

Tabla 1
Requerimientos funcionales

Id	Requerimiento	Actor	Cómo se comprueba
RF-01	Autenticación	Todos	Con credenciales válidas entra; con inválidas no; al cerrar sesión el token deja de servir
RF-02	Roles y permisos	Administrador	Un estudiante que llama una función de docente recibe "sin permiso" (403)
RF-03	Gestión de estudiantes	Administrador, coordinador	Se crea, edita y desactiva; el desactivado no se puede matricular
RF-04	Gestión de docentes	Administrador, coordinador	Igual al anterior; el desactivado no se puede asignar a un grupo
RF-05	Programas y asignaturas	Coordinador	No deja crear dos asignaturas con el mismo código
RF-06	Grupos y periodos	Coordinador	Se crea el grupo con cupo, docente, horario y aula o enlace
RF-07	Matrícula	Coordinador	Con cupo y sin duplicado matricula; si no, rechaza con el motivo
RF-08	Calificaciones	Docente	Solo registra notas en sus grupos
RF-09	Consulta de resultados	Estudiante	Ve su horario y solo las notas publicadas
RF-10	Reportes	Coordinador, administrador	El reporte por periodo y programa coincide con los datos
RF-11	Búsqueda y filtros	Usuarios autorizados	Busca por código, nombre, programa, periodo o estado según el rol
RF-12	Auditoría	Administrador	Cada login, cambio de rol, matrícula y cambio de nota queda registrado

4.2 Requerimientos no funcionales

Tabla 2
Requerimientos no funcionales

Id	Requerimiento	Cómo se mide
RNF-01	Seguridad	Todo por HTTPS, contraseñas con Argon2id, acceso por rol
RNF-02	Rendimiento	Con 300 usuarios, el 95 % de las consultas comunes en 3 s o menos
RNF-03	Disponibilidad	99,5 % mensual
RNF-04	Escalabilidad	Agregar servidores sin cambiar el código
RNF-05	Mantenibilidad	Cambiar una regla de notas solo toca el módulo de calificaciones
RNF-06	Usabilidad y accesibilidad	Se puede usar con teclado; buen contraste (WCAG 2.1 AA)
RNF-07	Integridad	Dos solicitudes por el último cupo dejan una sola matrícula
RNF-08	Interoperabilidad	API REST documentada con OpenAPI
RNF-09	Respaldo	Pérdida máxima de 15 min de datos y recuperación en menos de 2 h
RNF-10	Trazabilidad	Cada requerimiento aparece en la matriz con su componente
RNF-11	Privacidad	Un estudiante no puede ver los datos de otro
RNF-12	Compatibilidad	Funciona en Chrome, Edge, Firefox y Safari, en celular y computador

4.3 Reglas de negocio

Del análisis salieron reglas que el sistema tiene que validar en la lógica de negocio y no solo en la pantalla. Las más importantes son:

1. Un estudiante no puede quedar dos veces en el mismo grupo, ni en dos grupos de la misma asignatura en el mismo periodo.
2. No se puede matricular en un grupo sin cupo. Si llegan dos solicitudes al mismo tiempo por el último cupo, solo una puede quedar.
3. Solo se matricula en periodos abiertos y dentro de las fechas; solo estudiantes activos.
4. No se permite cruce de horario, ni para el estudiante ni para el docente.
5. El docente solo pone notas a estudiantes de sus grupos, entre 0,0 y 5,0.
6. Cuando el coordinador cierra las notas del periodo, el docente ya no puede cambiarlas.
7. Si se cambia una nota ya publicada, queda en la auditoría con el valor anterior, el nuevo y una justificación.
8. Cada quien ve solo lo suyo: el estudiante sus datos, el docente sus grupos.
9. En un grupo presencial el cupo no puede ser mayor que la capacidad del aula.
10. Los registros de auditoría no se pueden modificar ni borrar.

Algunas de estas reglas no las tenía al principio y salieron de las respuestas de la IA (las fechas, el cierre de notas, el cruce de horario y la capacidad del aula).

4.4 Análisis

Al revisar los requerimientos me di cuenta de que no todos pesan igual. Los que realmente definen la arquitectura son la disponibilidad (RNF-03), la mantenibilidad (RNF-05) y sobre todo la integridad de la matrícula (RNF-07); los demás se resuelven con medidas puntuales. El RNF-09 no daba cifras, así que propuse una pérdida máxima de 15 minutos de datos y 2 horas para recuperar el servicio. Al inicio había puesto 24 horas y 4 horas, pero lo bajé después de ver la propuesta de la IA, porque los servicios de base de datos administrados ya traen recuperación a un punto en el tiempo.

5. Arquitectura seleccionada y justificación

Escogí un **monolito modular en capas**. Es una sola aplicación, pero dividida en seis módulos según el área del negocio: identidad y acceso, gestión académica, matrículas, calificaciones, reportes y auditoría. Cada módulo tiene sus tres capas (controlador, dominio y repositorio), solo se comunica con los demás a través de interfaces y tiene su propio esquema dentro de una misma base de datos PostgreSQL. La aplicación se expone como una API REST y se despliega en dos servidores detrás de un balanceador.

5.1 Opciones que comparé

Comparé tres opciones: capas sin módulos, monolito modular y microservicios.

Tabla 3

Opciones que comparé

Criterio	Capas sin módulos	Monolito modular	Microservicios
Consistencia de la matrícula	Buena (una transacción)	Buena (una transacción)	Difícil: cada servicio con su base, riesgo de sobrecupo
Rendimiento con 300 usuarios	Suficiente	Bueno, llamadas en memoria	Latencia por la red
Mantenibilidad	Se va enredando con el tiempo	Buena, módulos separados	Buena por servicio, pero contratos complejos
Costo y operación	Bajo	Bajo	Alto

5.2 Por qué esta y no microservicios

La razón principal es la matrícula. Es el requisito más delicado: dos estudiantes no pueden quedarse con el mismo último cupo. Con una sola base de datos eso se resuelve con una transacción; con microservicios cada servicio tendría su base y habría que coordinarlos con consistencia eventual, lo que abre la puerta al sobrecupo.

Además, 300 usuarios concurrentes no es una carga tan grande: dos servidores alcanzan y se puede agregar un tercero sin tocar el código. Y para la mantenibilidad no hace falta partir el sistema en servicios; basta con que los módulos tengan límites claros. A eso se suma que el presupuesto es moderado y el equipo es pequeño.

No escogí solo capas porque las capas por sí solas no evitan que la lógica de matrícula, notas y auditoría termine mezclada. Los módulos ponen esos límites.

Si en el futuro algún módulo necesita crecer aparte, ya tiene su interfaz y su esquema, así que se podría sacar como un servicio poco a poco (patrón Strangler Fig).

La IA recomendó el mismo estilo cuando se lo pregunté, pero planteaba capas y módulos como opciones excluyentes. Para mí se complementan: módulos por fuera y capas por dentro.

6. Modelo del sistema

Hice los diagramas en Mermaid y el de casos de uso con un script que genera SVG; los archivos fuente están en el repositorio, en la carpeta `modelos/fuentes`. En todos usé los mismos nombres de actores, módulos y entidades.

6.1 Casos de uso

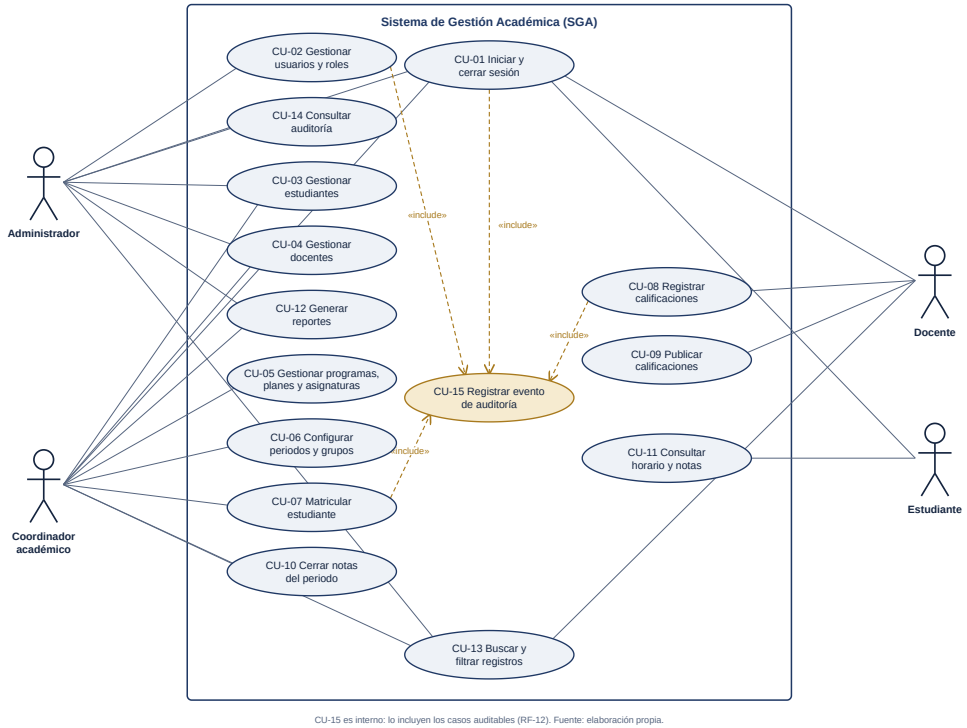
El diagrama muestra los cuatro actores y 15 casos de uso. Hay un caso interno, *registrar evento de auditoría*, que no tiene actor: lo incluyen siempre los casos que hay que auditar (iniciar sesión, gestionar roles, matricular y registrar notas), y así se cumple el RF-12.

Los casos más importantes son:

- **Matricular estudiante (CU-07):** el coordinador inscribe a un estudiante en uno o varios grupos; el sistema revisa cupo, duplicados, cruce de horario y fechas.
- **Registrar calificaciones (CU-08) y publicarlas (CU-09):** el docente pone las notas en borrador y luego las publica para que el estudiante las vea.
- **Cerrar notas del periodo (CU-10):** el coordinador bloquea la edición de notas.
- **Consultar horario y notas (CU-11):** el estudiante ve solo lo suyo.
- **Consultar auditoría (CU-14):** solo el administrador.

Figura 1

Diagrama de casos de uso

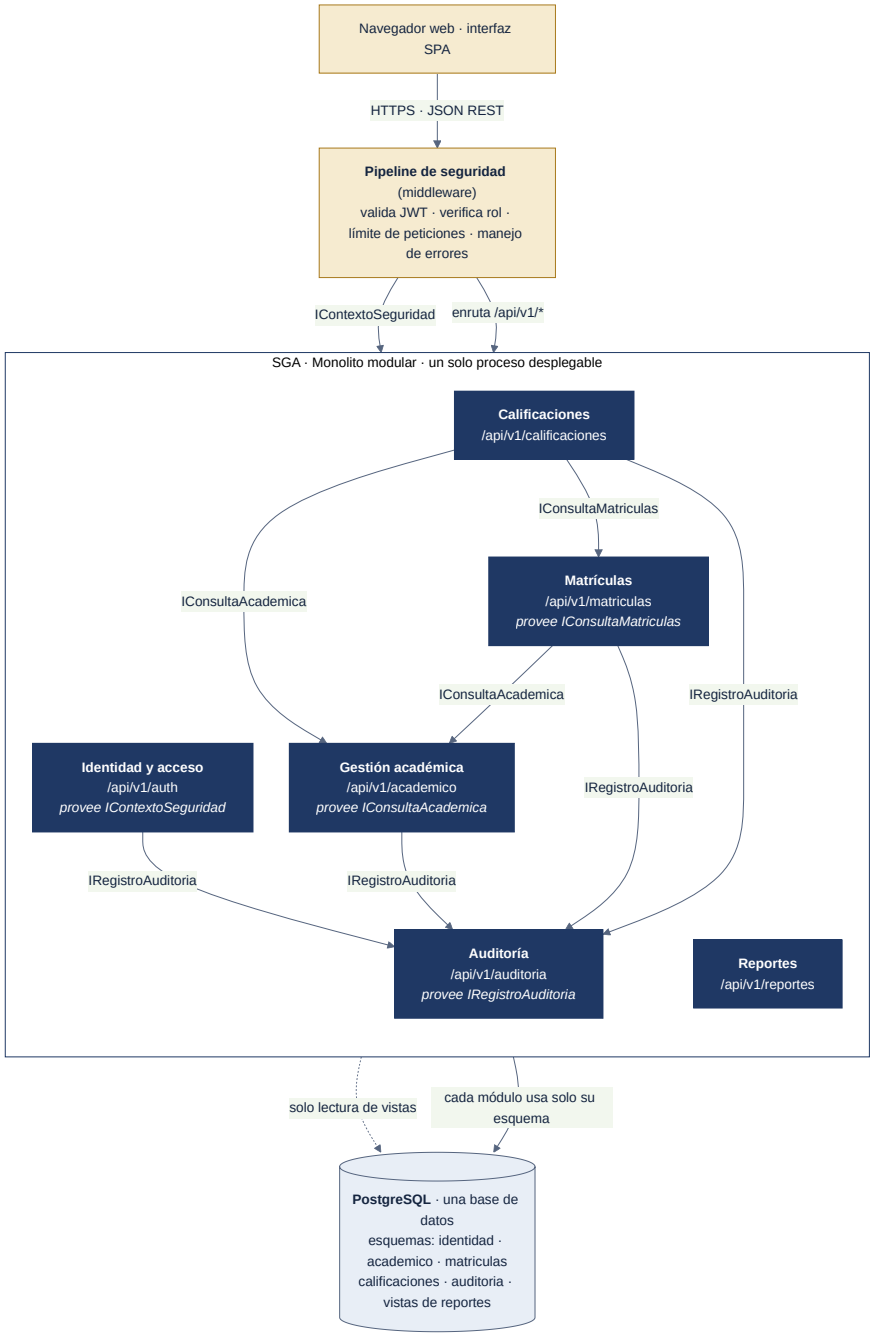


Nota. Elaboración propia.

6.2 Componentes (vista lógica)

En la figura 2 están los seis módulos con su ruta REST. Las flechas indican qué módulo usa a cuál, y la etiqueta es la interfaz que usa. Ningún módulo entra a las tablas de otro; si necesita algo, lo pide por la interfaz.

Figura 2
Módulos, interfaces y dependencias



Nota. Elaboración propia.

En resumen, cada módulo hace lo siguiente:

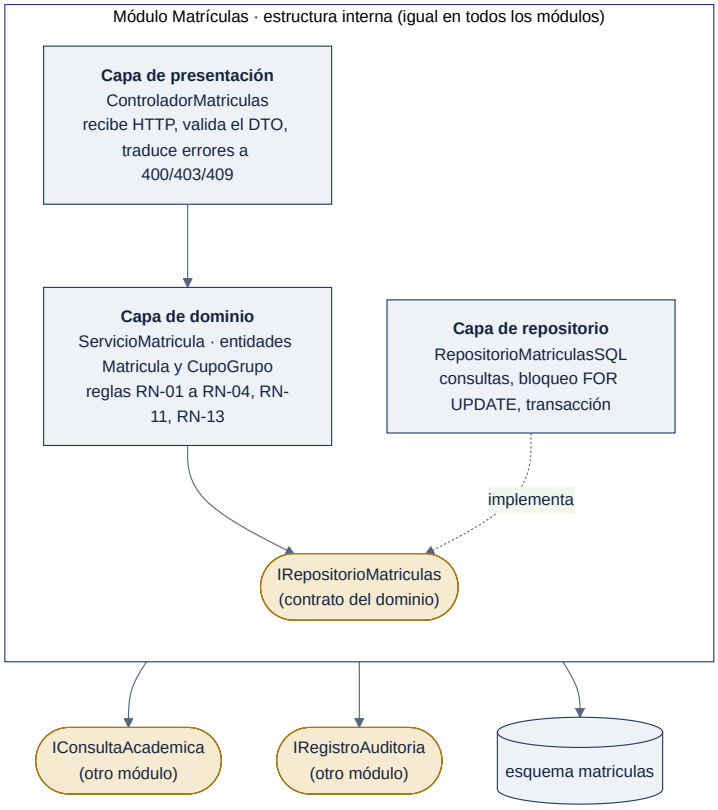
- **Identidad y acceso** maneja usuarios, contraseñas, roles e inicio y cierre de sesión. Le da a los demás la interfaz `IContextoSeguridad` (quién es el usuario y con qué rol está).
- **Gestión académica** maneja estudiantes, docentes, programas, planes, asignaturas, periodos, grupos, horarios y aulas. Expone `IConsultaAcademica` para que otros pregunten, por ejemplo, si un estudiante está activo.

- **Matrículas** hace la matrícula con todas las validaciones dentro de una transacción. Es dueño de la tabla de cupos y expone `IConsultaMatriculas`.
- **Calificaciones** registra, publica y corrige notas, revisando que el docente sea el del grupo y que las notas no estén cerradas.
- **Reportes** solo lee, a través de vistas de la base de datos con un usuario que no puede escribir.
- **Auditoría** guarda los eventos y solo los deja consultar al administrador. Expone `IRegistroAuditoria`.

La búsqueda y los filtros (RF-11) van en los listados de cada módulo, y siempre se aplica lo que el rol puede ver. Todos los endpoints se documentarían con OpenAPI.

Por dentro, todos los módulos tienen la misma estructura de tres capas (figura 3): el controlador recibe la petición, el dominio tiene las reglas y el repositorio habla con la base de datos. El dominio solo conoce el contrato del repositorio, no la base de datos, así que se podría cambiar la base sin tocar las reglas.

Figura 3
Estructura interna de un módulo (matrículas)



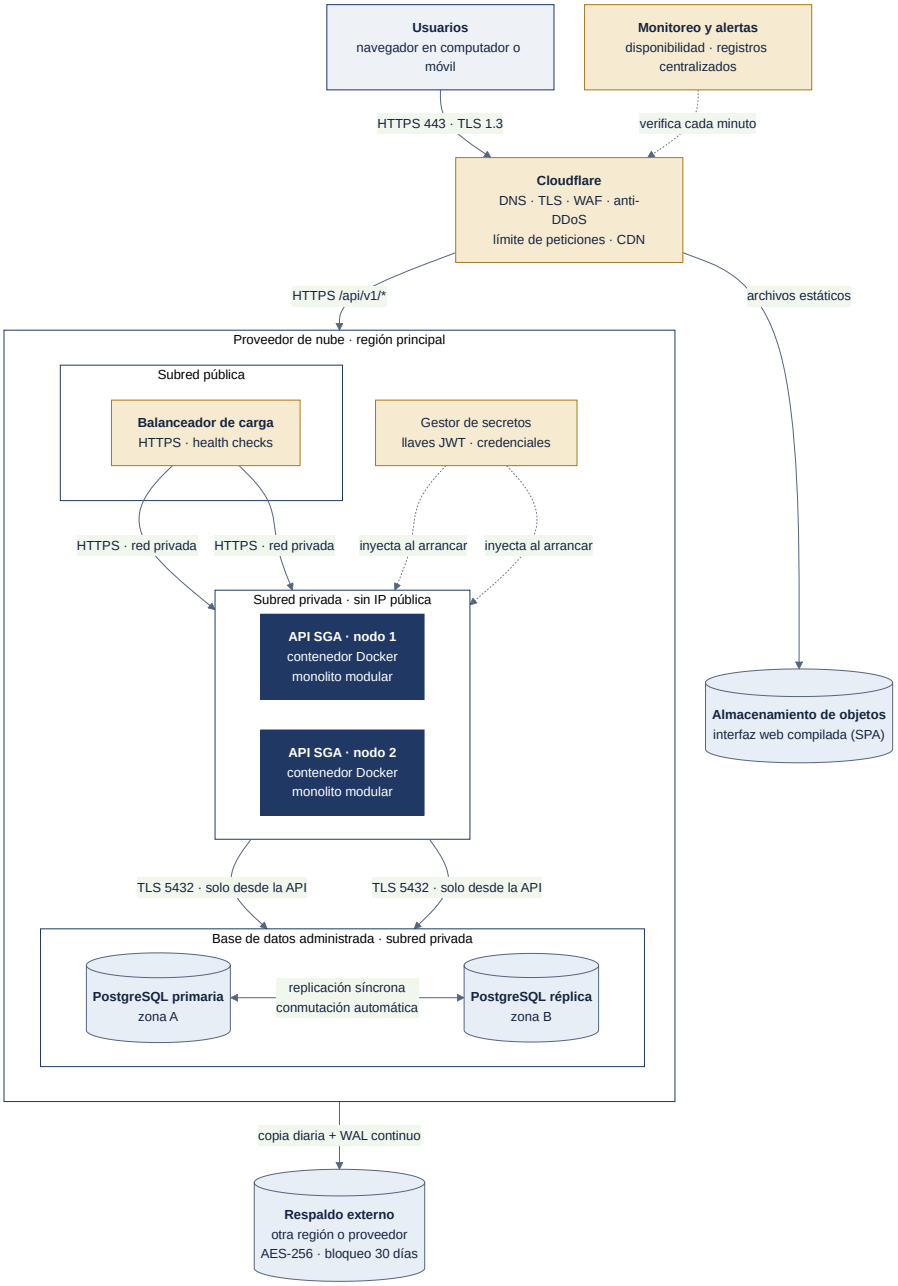
Nota. Elaboración propia.

6.3 Despliegue

Para el despliegue (figura 4) busqué que no hubiera un solo punto que tumbara todo el sistema, pero sin irme a algo costoso como Kubernetes en varias regiones.

Delante va Cloudflare, en su plan gratuito, que se encarga del certificado HTTPS, de frenar ataques y de servir la interfaz web. La API corre en dos contenedores Docker en una red privada, detrás del balanceador del proveedor. Si uno se cae, el otro sigue atendiendo. La base de datos es un PostgreSQL administrado con una réplica en otra zona que toma el control sola si falla la principal. Los respaldos se hacen todos los días y además se guarda el registro continuo de cambios (WAL), cifrado y en otra región. Las contraseñas de la base y las llaves de los tokens no van en el código: se guardan en un gestor de secretos. Y hay un monitor externo que avisa si el sistema se cae.

Figura 4
Nodos, canales y seguridad



Nota. Elaboración propia.

6.4 Modelo de datos

La figura 5 muestra las entidades y cómo se relacionan. Cada una pertenece al esquema de un módulo: Usuario, Rol y Sesión a identidad; Estudiante, Docente, Programa, Plan de estudio, Asignatura, Periodo, Grupo, Horario y Aula a gestión académica; Matrícula, Detalle de matrícula y Cupo de grupo a matrículas; Evaluación y Calificación a calificaciones; y Evento de auditoría a auditoría.

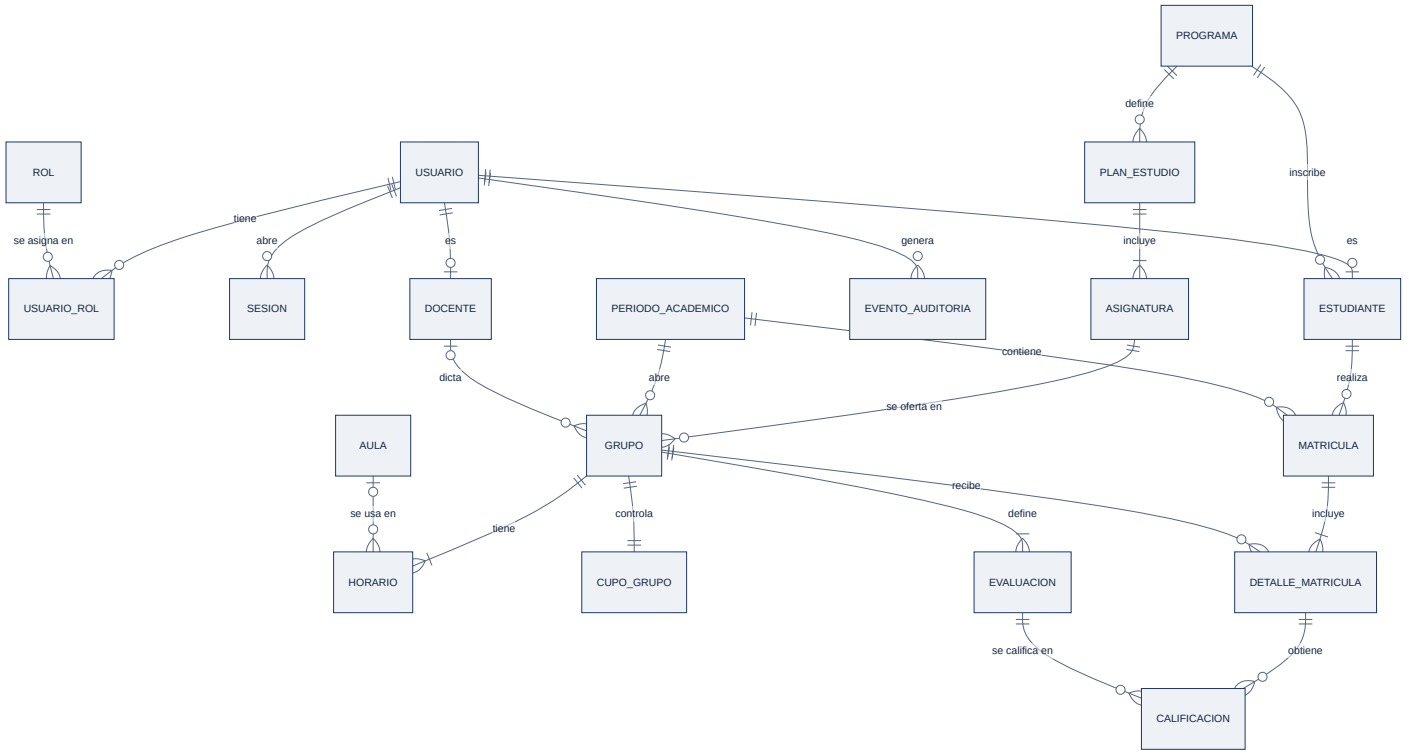
Algunas relaciones que vale la pena explicar:

- Usuario y Rol es de muchos a muchos, porque un mismo usuario puede tener varios roles.
- La matrícula es una por estudiante y periodo, y tiene un detalle por cada grupo.
- Cada grupo tiene sus evaluaciones (por ejemplo, Parcial 1 con 30 %) y la nota de cada estudiante se guarda por evaluación.
- El cupo ocupado no está en Grupo sino en Cupo de grupo, que es del módulo de matrículas.

Para que no haya duplicados ni sobrecupo, aunque falle el código, la base de datos tiene restricciones: una matrícula por estudiante y periodo, no repetir grupo ni asignatura dentro de la misma matrícula, y que los ocupados nunca pasen del cupo máximo. Los atributos completos de cada entidad están en el modelo detallado del repositorio.

Figura 5

Entidades principales y cardinalidades



Nota. Notación de pata de gallo. Elaboración propia.

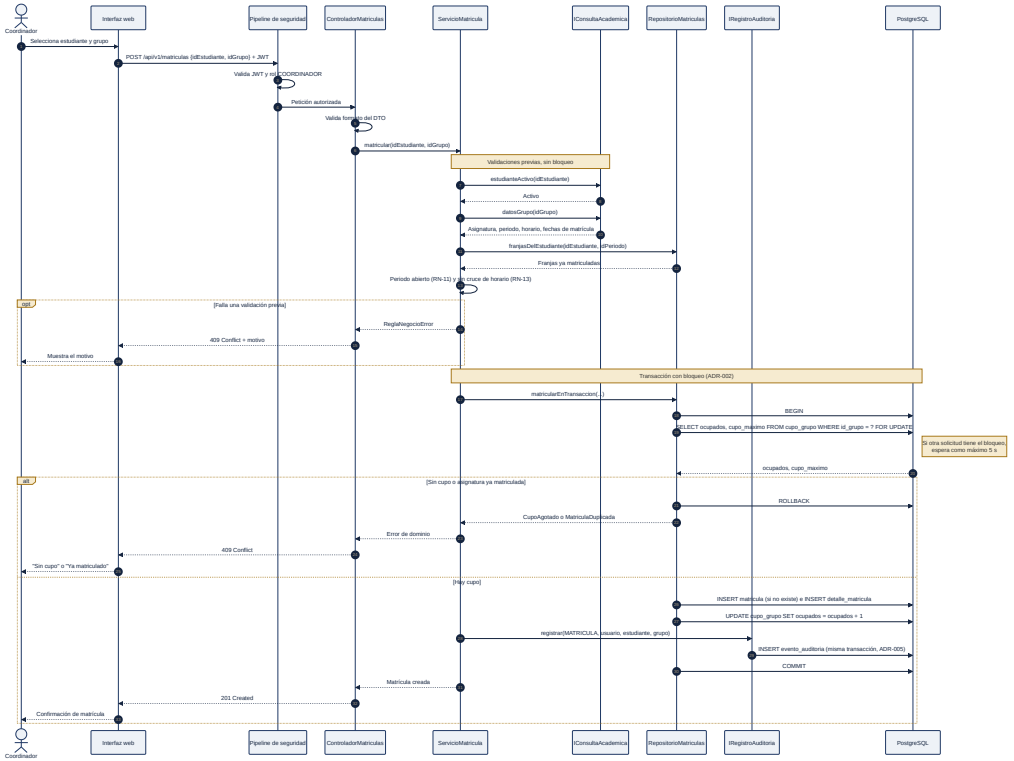
6.5 Secuencia de la matrícula

Escogí modelar la matrícula porque es el proceso más delicado. En la figura 6 se ve el flujo completo:

Primero el pipeline de seguridad revisa el token y el rol. Después, antes de bloquear nada, el sistema valida lo que no depende del cupo: que el estudiante esté activo, que el periodo esté abierto y que no haya cruce de horario. Ahí sí abre una transacción y bloquea solo la fila del cupo de ese grupo (`SELECT ... FOR UPDATE`). Si no hay cupo o ya tiene la asignatura, deshace todo y responde con un error 409. Si hay cupo, guarda la matrícula, suma uno a los ocupados y escribe la auditoría, todo en la misma transacción.

Figura 6

Flujo de la matrícula

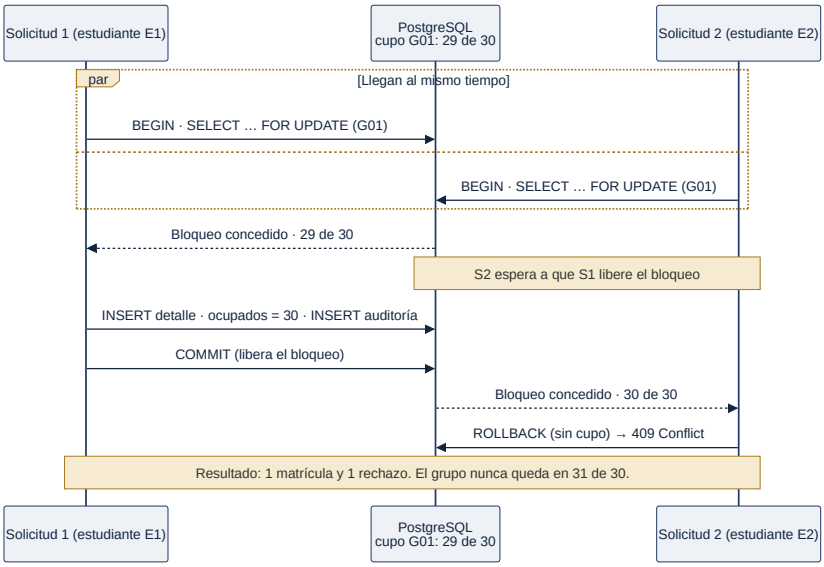


Nota. Elaboración propia.

La figura 7 muestra el caso que más me preocupaba: dos solicitudes al mismo tiempo por el último cupo. La primera toma el bloqueo; la segunda espera. Cuando la primera termina, la segunda ve que ya no hay cupo y se rechaza. El grupo nunca queda con 31 de 30. Cuando se matriculan varios grupos a la vez, los bloqueos se toman en orden para evitar que dos solicitudes se queden esperándose entre sí; esa idea la tomé de la IA.

Figura 7

Dos solicitudes por el último cupo



Nota. Elaboración propia.

7. Matriz de trazabilidad

En esta matriz relaciono cada requerimiento con el módulo que lo cumple, los modelos donde aparece y cómo se comprobaría. Como no se implementó el sistema, la última columna es la prueba que propongo, no un resultado.

Tabla 4
Matriz de trazabilidad

Req.	Componente	Modelos	Decisión	Cómo se comprobaría
RF-01	Identidad y acceso	CU-01, componentes, datos	ADR-004	Prueba de login válido, inválido y cierre de sesión
RF-02	Identidad y acceso, cada módulo	CU-02, componentes	ADR-004	Llamar cada servicio con los 4 roles y revisar los 403
RF-03	Gestión académica	CU-03, datos	—	Desactivar un estudiante e intentar matricularlo
RF-04	Gestión académica	CU-04, datos	—	Desactivar un docente e intentar asignarlo
RF-05	Gestión académica	CU-05, datos	—	Crear dos asignaturas con el mismo código
RF-06	Gestión académica, matrículas	CU-06, datos	—	Grupo con cruce de horario o cupo mayor que el aula
RF-07	Matrículas	CU-07, secuencia, datos	ADR-002	Pruebas de cada regla y prueba de concurrencia
RF-08	Calificaciones	CU-08, CU-10, datos	—	Nota en grupo ajeno, nota de 5,5, nota después del cierre
RF-09	Calificaciones	CU-11, datos	—	El estudiante no ve notas en borrador
RF-10	Reportes	CU-12, componentes	—	Comparar el reporte con un conteo directo
RF-11	Todos los listados	CU-13	—	Un docente solo encuentra sus grupos
RF-12	Auditoría	CU-14, CU-15, secuencia	ADR-005	Matar el proceso en medio de matrículas y comparar los conteos
RNF-01	Cloudflare, identidad, red privada	Despliegue	ADR-003, ADR-004	Escaneo con OWASP ZAP; revisar que solo haya hashes
RNF-02	API, base de datos	Despliegue	ADR-001	Prueba de carga con 300 usuarios (k6)
RNF-03	Balanceador, 2 nodos, réplica	Despliegue	ADR-003	Apagar un nodo y forzar el cambio de base de datos
RNF-04	API sin estado	Despliegue	ADR-004	Agregar un tercer nodo sin cambiar código
RNF-05	Módulos con interfaces	Componentes	ADR-001	Pruebas de arquitectura (ArchUnit)
RNF-06	Interfaz web	—	—	Lighthouse y axe; usar el sistema solo con teclado
RNF-07	Matrículas, restricciones	Secuencia, datos	ADR-002	50 solicitudes por 1 cupo: debe quedar 1 matrícula
RNF-08	API REST	Componentes	—	Validar la especificación OpenAPI
RNF-09	Base de datos, respaldo	Despliegue	ADR-003	Simulacro de restauración cada tres meses
RNF-10	Esta matriz, ADR, auditoría	Todos	—	Revisar que todo requerimiento tenga fila
RNF-11	Filtros por rol en cada módulo	Componentes	ADR-004	El estudiante A pide datos del B y recibe 403
RNF-12	Interfaz web	Despliegue	—	Probar en 4 navegadores y 2 tamaños de pantalla

Los requerimientos que quedan más débiles son la accesibilidad y la compatibilidad, porque dependen de cómo se haga la interfaz, que no está en el alcance.

8. Decisiones arquitectónicas y análisis de riesgos

8.1 Decisiones

Documenté cinco decisiones con el formato ADR (contexto, decisión, opciones y consecuencias), siguiendo a Nygard (2011). El texto completo de cada una está en el repositorio, en `docs/adr`.

ADR-001. Monolito modular en capas. Lo expliqué en la sección 5. La consecuencia negativa es que el sistema crece completo y no por partes, y que los límites entre módulos dependen de la disciplina del equipo.

ADR-002. Bloqueo de fila en la matrícula. Para que no haya sobrecupo, la matrícula bloquea la fila del cupo del grupo dentro de una transacción. Consideré también el control optimista, pero con mucha gente peleando el último cupo genera muchos reintentos fallidos, y una cola de solicitudes, que agregaba otro componente sin necesidad. Lo malo es que un grupo muy pedido hace fila; por eso las validaciones que no dependen del cupo van antes del bloqueo y la espera máxima es de 5 segundos.

ADR-003. Despliegue con réplicas y respaldo externo. Dos servidores para la API, base de datos con réplica en otra zona y respaldo continuo. Un solo servidor era más barato pero no alcanza el 99,5 %, y Kubernetes en varias regiones era demasiado para 300 usuarios. Queda la dependencia de proveedores externos que manejan datos personales.

ADR-004. Inicio de sesión sin estado. Un token de 15 minutos y otro de renovación de 8 horas que cambia cada vez que se usa. Así cualquier servidor atiende cualquier petición. Lo malo es que después de cerrar sesión el token corto sigue sirviendo hasta 15 minutos.

ADR-005. Auditoría en la misma transacción. El registro de auditoría se guarda en la misma transacción del cambio, así que si el cambio queda guardado, la auditoría también. La IA proponía hacerlo de forma asíncrona, pero así se podían perder registros si el servidor se caía en el momento justo.

8.2 Atributos de calidad

En pocas palabras, cada atributo lo atiendo así: el rendimiento con llamadas en memoria, índices y un pool aparte para los reportes; la disponibilidad con redundancia y cambio automático de base de datos; la integridad con la transacción y las restricciones; la recuperación con los respaldos; la seguridad con HTTPS, Argon2id, roles y red privada; y la mantenibilidad con los módulos y sus interfaces.

8.3 Riesgos y puntos únicos de falla

Revisé componente por componente qué pasaría si fallara. Los nodos de la API, el balanceador y la base de datos están duplicados, así que no tumban el sistema. Redis y el bus de eventos no existen en mi diseño. Quedan dos puntos únicos de falla que acepto: Cloudflare, que si falla obligaría a apuntar el DNS directo al balanceador, y la región del proveedor de nube, que si cae completa se recuperaría desde el respaldo en otra región en unas 2 horas.

Califiqué los riesgos con probabilidad por impacto, de 1 a 3 cada uno:

Tabla 5

Califiqué los riesgos con probabilidad por impacto, de 1 a 3 cada uno

Riesgo	Nivel	Qué hago al respecto
Sobrecupo o matrícula duplicada	Alto (9)	Bloqueo de fila, restricciones y prueba de concurrencia
Un cambio sin auditoría	Alto (6)	Auditoría en la misma transacción y prueba matando el proceso
Se agotan las conexiones a la base	Alto (6)	Límite de conexiones por servidor y pool aparte para reportes
Un estudiante ve datos de otro	Alto (6)	Filtros por rol en cada módulo
Datos personales en proveedores sin contrato	Alto (6)	Revisión jurídica (Ley 1581 de 2012) antes de salir a producción
Fila larga si se habilita la automatrícula	Medio (4)	Rechazo inmediato si el grupo ya está lleno
Los reportes ponen lento el sistema	Medio (4)	Vistas de solo lectura y pool aparte
Usuario con dos roles usa permisos del otro	Medio (4)	Un solo rol activo en el token
Robo de token en un computador compartido	Medio (4)	Tokens cortos que rotan
Los módulos se van mezclando	Medio (4)	Pruebas de arquitectura
El superusuario de la base altera la auditoría	Medio (3)	Copia diaria de la auditoría en almacenamiento bloqueado
El respaldo no sirve cuando se necesita	Medio (3)	Simulacro de restauración

8.4 Límites y validación

El sistema crece completo y no por módulo, la base principal recibe todas las escrituras y un grupo muy pedido siempre va a hacer fila. Para 300 usuarios no es un problema, pero son los límites a tener en cuenta.

Las decisiones que habría que comprobar antes de construir el sistema son: una prueba de carga subiendo de 50 a 500 usuarios (70 % consultas, 20 % matrículas y 10 % reportes), una prueba de 300 usuarios peleando 5 cupos en 2 segundos, apagar un servidor y forzar el cambio de base de datos, un simulacro de restauración, y matar el proceso en medio de matrículas para revisar que no se pierda ninguna auditoría. Varias de estas pruebas las propuso la misma IA en su autocrítica.

9. Uso de Google AI Studio

9.1 Cómo lo usé

Usé el Playground de AI Studio con Gemini 3.8 Flash, con razonamiento en alto y la búsqueda en Google apagada para que solo trabajara con el caso. Todo fue el 4 de noviembre de 2026, en una sola conversación. El enlace a la conversación original es: https://aistudio.google.com/app/prompts?state=%7B%22ids%22:%5B%221NcbJ3YkHGzE2xID9EykCrpI4RXmgqqe8%22%5D,%22action%22:%22open%22,%22userId%22:%221137483483827505069499 5%2

Antes de abrir AI Studio hice mi propio análisis de los requerimientos y tenía una idea del estilo que quería. Lo hice así a propósito, para tener contra qué comparar lo que respondiera. Después le mandé siete prompts en el orden en que se diseña una arquitectura, sin corregirlo entre uno y otro:

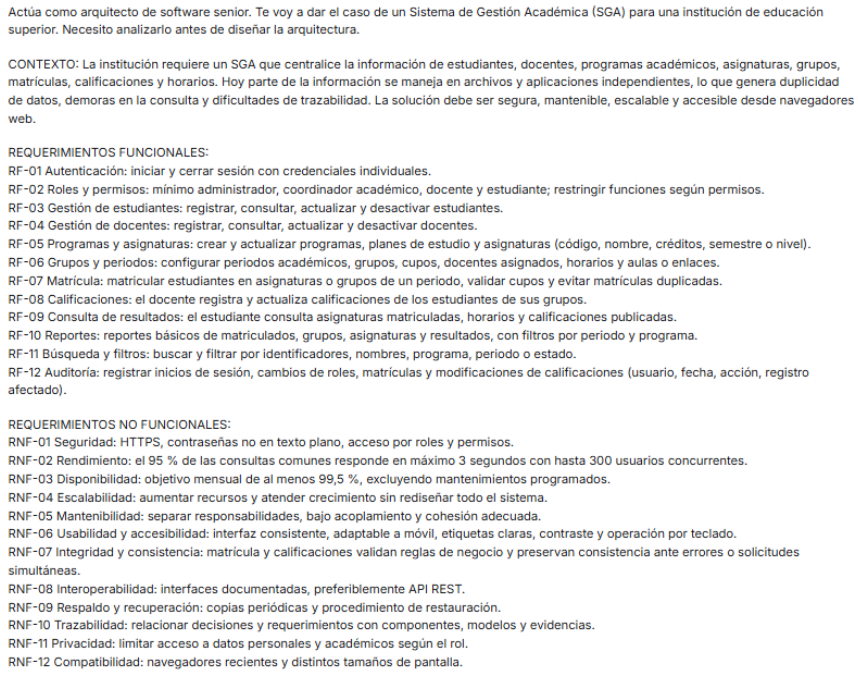
1. Análisis del problema: le di el caso y los 24 requerimientos y le pedí actores, supuestos, restricciones y reglas.
2. Estilo arquitectónico: que comparara capas, monolito modular y microservicios.
3. Componentes: módulos, interfaces y un diagrama.
4. Modelo de datos.
5. Secuencia de la matrícula, incluyendo dos solicitudes por el último cupo.
6. Despliegue, seguridad y respaldo con presupuesto moderado.
7. Que criticara su propia propuesta como evaluador externo.

El texto de cada prompt está en el anexo C y las capturas en el anexo B.

Además de AI Studio, me apoyé en Claude (Anthropic) para organizar la redacción del documento y dibujar los diagramas en Mermaid. Las decisiones y la revisión de las respuestas de Gemini las hice contrastándolas con los requerimientos del caso.

Figura 8

Prompt P1 enviado a Gemini



Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión Académica (SGA) para una institución de educación superior. Necesito analizarlo antes de diseñar la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes, programas académicos, asignaturas, grupos, matrículas, calificaciones y horarios. Hoy parte de la información se maneja en archivos y aplicaciones independientes, lo que genera duplicidad de datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura, mantenible, escalable y accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante; restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas (código, nombre, créditos, semestre o nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados, horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones (usuario, fecha, acción, registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300 usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras, contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y preservan consistencia ante errores o solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

Nota. Captura de Google AI Studio.

9.2 Qué hice con las respuestas

Revisé cada propuesta de cada respuesta y la clasifiqué como aceptada, modificada o descartada. En total fueron 93 propuestas:

Tabla 6

Revisé cada propuesta de cada respuesta y la clasifiqué como aceptada, modificada o descartada. En total fueron 93 propuestas

Prompt	Aceptadas	Modificadas	Descartadas
P1 Análisis	11	6	1
P2 Estilo	5	5	0
P3 Componentes	5	9	1
P4 Datos	6	7	1

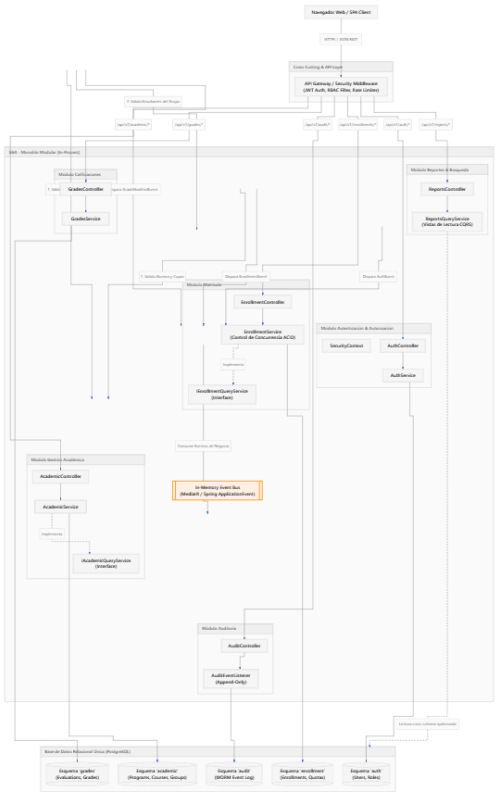
Prompt	Aceptadas	Modificadas	Descartadas
P5 Secuencia	2	7	0
P6 Despliegue	10	4	1
P7 Autocrítica	8	2	2
Total	47	40	6

El detalle de cada una está en el anexo A. Lo más importante que corregí fue:

- **La auditoría.** En el P3 y el P5 la IA propuso guardarla de forma asíncrona, después de confirmar el cambio. Si el servidor se cae justo en medio, el cambio queda sin registro, y eso va contra el RF-12. Lo cambié a la misma transacción. En el P7 la propia IA reconoció ese riesgo.
- **El aislamiento entre módulos.** La IA dijo que ningún módulo debía tocar las tablas de otro, pero después, en la matrícula, bloqueaba la tabla de grupos, que es de gestión académica. Lo resolví con una tabla de cupos propia del módulo de matrículas. Lo repitió en el P3, el P4, el P5 y otra vez en el prototipo.
- **Cosas de más.** Propuso Redis cuatro veces, además de almacenamiento WORM, colas, un servicio de notificaciones y ABAC. Ninguno lo pedía un requerimiento, así que los quité o los simplifiqué.
- **Cosas que faltaban.** En el modelo de datos no estaban los planes de estudio, el estado de publicación de las notas ni las fechas del periodo, y el porcentaje de cada evaluación se repetía en cada nota.
- **Los diagramas.** Tres de los cuatro que generó salieron ilegibles (figura 9), así que los rehice todos.

También tomé ideas buenas que yo no tenía: el cierre de notas, el cruce de horario, el orden de los bloqueos, la recuperación a un punto en el tiempo, la capacidad del aula, el rol activo y el plan de pruebas.

Figura 9
Diagrama de componentes que generó la IA (P3)



Nota. Captura de Google AI Studio. Por esto rehice el diagrama (figura 2).

9.3 Prototipo en AI Studio Build

Después usé AI Studio Build para generar un prototipo navegable. En lugar de pedirle "un sistema académico", le pasé mi arquitectura: los seis módulos con sus tres capas en carpetas separadas, las interfaces, las reglas de negocio, el rol activo, la auditoría y una página con las cinco decisiones. También le pedí un botón que simulara dos solicitudes por el último cupo. El prompt está en el repositorio (`docs/ai-studio/PROMPT-BUILD.txt`).

El prototipo salió con los cuatro perfiles, el menú según el rol (por ejemplo, el coordinador no ve la auditoría), la baja lógica de estudiantes, la matrícula con sus cinco validaciones y la simulación del último cupo (figuras 10 y 11).

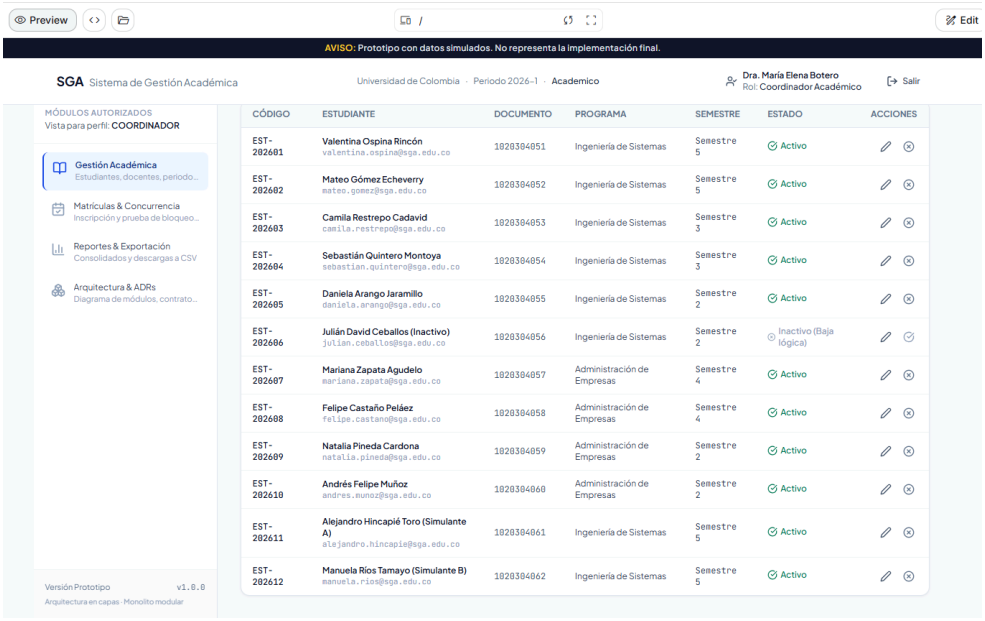
Pero al revisarlo encontré dos cosas:

- En la explicación del botón de concurrencia, la IA volvió a bloquear la tabla `grupos`, que es del módulo académico, en vez de la tabla de cupos del módulo de matrículas. Es el mismo error que ya le había corregido en el diseño, así que se lo volví a corregir.
- Gemini dice que la auditoría es "transaccional ACID" y que la simulación prueba la concurrencia. No es así: el prototipo guarda los datos en memoria del navegador, donde no hay transacciones, y JavaScript corre en un solo hilo, así que las dos solicitudes en realidad se procesan una tras otra. El botón sirve para ilustrar la idea, pero la garantía real la da PostgreSQL.

Por eso insisto en que el prototipo es una maqueta con datos simulados y no la implementación de la arquitectura.

Figura 10

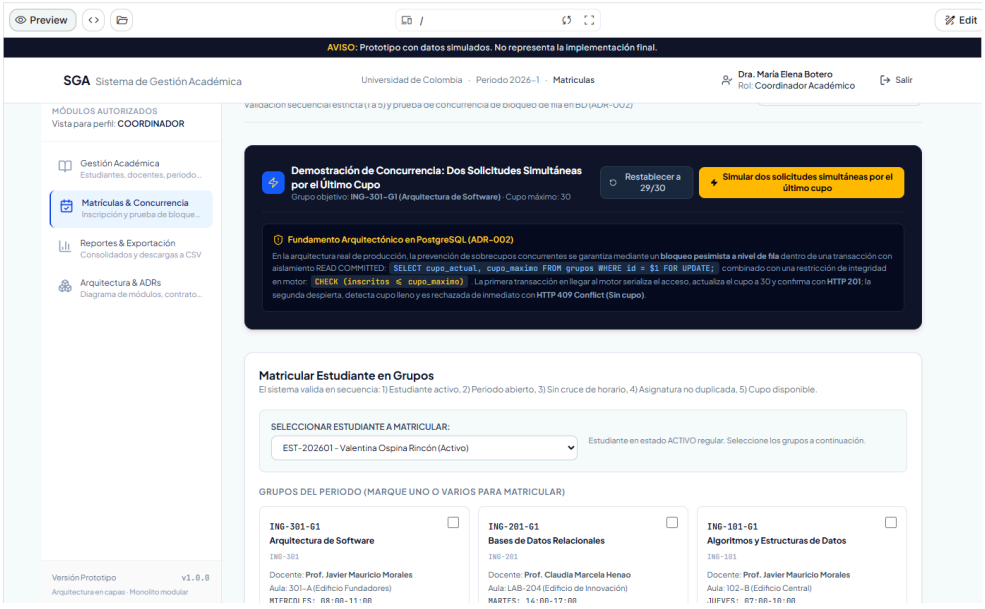
Prototipo: listado de estudiantes como coordinador



Nota. Captura de AI Studio Build. Se ve la baja lógica y el menú sin auditoría.

Figura 11

Prototipo: matrícula y explicación del bloqueo



Nota. Captura de AI Studio Build. En el recuadro la IA bloquea la tabla grupos, que es de otro módulo.

9.4 Lo que aprendí usando la IA

La IA acertó en lo grande: recomendó el mismo estilo, los mismos módulos y el bloqueo de fila, y su plan de pruebas fue bueno. También me dio ideas que no tenía. Pero tendió a meter cosas de más, se contradijo (defendía el aislamiento entre módulos y luego lo rompía) y repitió errores que ya le había corregido. Por eso no basta con corregirla una vez: cada respuesta hay que revisarla contra los requerimientos.

10. Conclusiones

1. Un monolito modular en capas cumple los 24 requerimientos con menos complejidad y costo que los microservicios. La decisión sale de los requerimientos, sobre todo de la integridad de la matrícula, y no de qué tecnología está de moda.
2. La matrícula es la parte más delicada del sistema porque junta reglas de negocio, concurrencia y auditoría. Con una transacción, el bloqueo de fila y las restricciones en la base de datos no puede haber sobrecupo aunque falle el código.
3. El 99,5 % de disponibilidad se logra duplicando lo que podía tumbar el sistema (servidores, balanceador y base de datos). Quedan dos riesgos que acepto: Cloudflare y la región de la nube.
4. Todos los requerimientos quedaron con su componente, su modelo y una forma de comprobarlos. Los más débiles son la accesibilidad y la compatibilidad, que dependen de la interfaz.
5. La IA me ahorró mucho tiempo pero no reemplazó el trabajo de pensar. Lo útil fue tener mi propio análisis antes de consultarla y revisar cada respuesta.

Como recomendación, antes de construir el sistema habría que hacer las pruebas de la sección 8, confirmar los supuestos con la institución (sobre todo quién matricula y los tiempos de respaldo) y revisar los contratos con los proveedores por el manejo de datos personales.

Enlaces

- Video: https://youtu.be/XETzIcJ7SuE
- Repositorio: https://github.com/juanvs1985/sga-arquitectura

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.° 48.587.

Google. (2026). *Gemini 3.8 Flash* [Modelo de lenguaje de gran tamaño]. Google AI Studio. https://aistudio.google.com

International Organization for Standardization. (2011). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models* (ISO/IEC 25010:2011).

Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico* (9.ª ed.). McGraw-Hill Education.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Anexo A. Evaluación de cada respuesta de la IA

P1 · análisis del problema

#	Propuesta de la IA	Decisión	Justificación
1.1	Cuatro actores principales: estudiante, docente, coordinador académico, administrador.	Aceptado	Coincide con RF-02 y con el análisis propio.
1.2	El **estudiante realiza su propia matrícula**.	Modificado	RF-07 no dice quién matricula. Mantenemos al coordinador como responsable (supuesto S-02) para reducir el riesgo de concurrencia masiva en esta primera versión. La autoinscripción queda como evolución: la API de matrícula es la misma y solo cambia el permiso.
1.3	**Servicio de auditoría** como actor secundario.	Modificado	La auditoría es un módulo interno del sistema (RF-12), no un actor externo. Se modela como componente, no en el diagrama de casos de uso.
1.4	Servicio de copias de seguridad como actor secundario.	Aceptado	Coincide con el análisis propio (RNF-09).
1.5	**Servicio de notificaciones** (correo/SMS).	Descartado	Ningún RF lo pide. Agregarlo amplía el alcance sin requisito que lo justifique. Se menciona como recomendación futura.
1.6	Baja lógica (soft delete) al desactivar estudiantes y docentes.	Aceptado	Igual al supuesto S-03; preserva el historial y la integridad referencial.
1.7	**Ventanas de tiempo** para matrícula y registro de notas.	Aceptado	Amplía nuestra regla RN-03 (periodo abierto). Se agrega como RN-11.
1.8	Calificaciones ponderadas en escala 0,0–5,0 con nota aprobatoria configurable.	Aceptado	Coincide con RN-06.
1.9	**Validar cruces de horario** de estudiantes y docentes.	Aceptado	Buena regla que el análisis propio no tenía. Se agrega como RN-13.
1.10	Control de concurrencia en la matrícula: optimista, pesimista **o colas**.	Modificado	Se elige bloqueo pesimista de la fila del grupo dentro de una transacción, más una restricción única en la base. Las colas se descartan: con 300 usuarios concurrentes agregan complejidad sin beneficio (ver ADR-002).
1.11	Desacoplar el proveedor de identidad para un SSO futuro (OAuth2, SAML, Google o Microsoft).	Modificado	Se acepta como punto de extensión del módulo Identidad y acceso, pero queda fuera del alcance actual: RF-01 pide credenciales propias.
1.12	Restricciones técnicas: navegador, API REST con OpenAPI, TLS, hash Argon2id o bcrypt, 3,6 h de caída al mes.	Aceptado	Coinciden con R-01 a R-05.
1.13	Caché con **Redis** para catálogos y **CDN** para estáticos.	Pendiente (se decide en P2)	Hay que validar si se justifican para 300 usuarios y presupuesto moderado.
1.14	Auditoría **WORM** (almacenamiento de una sola escritura).	Modificado	WORM es costoso para el presupuesto supuesto (S-08). Se cumple el mismo objetivo con una tabla de auditoría de solo inserción: la cuenta de la aplicación no tiene permisos de UPDATE ni DELETE sobre ella.
1.15	Cierre y **congelamiento de calificaciones**: después del cierre, el docente no edita; cualquier cambio requiere autorización con auditoría.	Aceptado	Regla valiosa que el análisis propio no tenía. Se agrega como RN-12.
1.16	Auditoría inmutable (solo adición).	Aceptado	Se agrega como RN-14, implementada como se indica en 1.14.
1.17	Control de acceso **ABAC/RBAC**.	Modificado	Se usa RBAC (RF-02) más filtros de propiedad en el dominio ("solo lo mío", "solo mis grupos"). ABAC completo no se justifica en este alcance.
1.18	Clasificación de los RNF según ISO/IEC 25010.	Aceptado	Coincide con la tabla 1.7 del análisis. Se adopta la subcaracterística "Recuperabilidad" para RNF-09.
1.19	IDs de correlación en las trazas (RNF-10).	Aceptado	Mejora la trazabilidad técnica de cada petición entre los módulos y la auditoría.

Balance P1: 11 aceptadas, 6 modificadas, 1 descartada, 1 pendiente. La IA aportó tres reglas de negocio que el análisis propio no tenía: ventanas de tiempo, cierre de calificaciones y cruces de horario. En cambio, tendió a sobredimensionar la solución (WORM, colas, notificaciones, ABAC) frente al tamaño del problema.

P2 · estilo arquitectónico

#	Propuesta de la IA	Decisión	Justificación
2.1	Recomendar **monolito modular** con una base de datos relacional y API REST.	Aceptado	Coincide con la decisión que el equipo ya había tomado antes de consultar a la IA. Se documenta en ADR-001 con argumentos propios: tamaño del sistema, 300 usuarios concurrentes, consistencia de la matrícula y un solo desarrollador.
2.2	Presentar "capas" y "monolito modular" como **alternativas excluyentes**, y advertir que las capas terminan en una "gran bola de lodo".	Modificado	No son excluyentes. La propuesta final combina ambos: módulos verticales por contexto de negocio y, dentro de cada módulo, capas de presentación (controlador), dominio y repositorio. El riesgo de la gran bola de lodo viene de no tener límites entre módulos, no de usar capas.
2.3	Descartar microservicios por complejidad operativa, latencia de red y consistencia eventual (Sagas) en la matrícula.	Aceptado	Argumento correcto y verificable. Además, el equipo es de una persona, lo que refuerza el descarte.
2.4	Bloqueo pesimista (`SELECT ... FOR UPDATE`) **o** control optimista para los cupos.	Modificado	Se elige el bloqueo pesimista de la fila del grupo más una restricción única. Con contención alta por el último cupo, el control optimista produce muchos reintentos fallidos (ver ADR-002).
2.5	**Partición lógica de esquemas**: una base de datos con un esquema por módulo.	Aceptado	Refuerza el bajo acoplamiento: cada módulo es dueño de sus tablas y los demás solo acceden por su interfaz pública. Facilita extraer un módulo en el futuro.
2.6	2 o 3 réplicas en contenedores Docker detrás de un balanceador para el 99,5 %.	Aceptado	Se fijan 2 réplicas como mínimo. Requisito previo: la API debe ser sin estado (sesión en token, no en memoria).
2.7	Auditoría mediante **interceptores (middleware)**.	Modificado	Un middleware no conoce el valor anterior y el nuevo de una nota. Se usa middleware para los inicios de sesión, y para matrículas, notas y roles el módulo de dominio escribe el registro de auditoría **dentro de la misma transacción** del cambio: si el cambio se guarda, la auditoría también.
2.8	Afirmaciones como rendimiento "excelente" o "supera con holgura los 3 s".	Modificado	Son afirmaciones no verificadas. Se convierten en hipótesis que se comprobarán con una prueba de carga de 300 usuarios virtuales (ver matriz de trazabilidad, RNF-02).
2.9	Si un módulo crece, extraerlo a microservicio en el futuro.	Aceptado	Se documenta como camino de evolución con el patrón Strangler Fig.
1.13	(Pendiente del P1) Caché con **Redis** y **CDN**.	Redis: Descartado · CDN: Modificado	Redis: con 300 usuarios y sesión en token no hay estado compartido que guardar; los catálogos (programas, asignaturas) se cachean en memoria de cada réplica. Se reconsidera si la prueba de carga no cumple RNF-02. CDN: la interfaz web estática se publica en un hosting que ya incluye distribución por CDN, sin costo ni componente adicional.

Balance P2: 5 aceptadas, 5 modificadas (incluido el pendiente del P1). La recomendación de la IA coincide con la del equipo, pero la propuesta final corrige tres puntos: capas y módulos se combinan, la auditoría va en la misma transacción y el rendimiento se trata como una hipótesis por verificar.

P3 · componentes y vista lógica

#	Propuesta de la IA	Decisión	Justificación
3.1	Seis módulos: autenticación y autorización, gestión académica, matrícula, calificaciones, reportes y búsqueda, auditoría.	Aceptado	Coinciden con los seis módulos de ADR-001. En el modelo final se nombran en español para mantener la consistencia con las entidades del enunciado.
3.2	Interfaces internas `IAcademicQueryService` e `IEnrollmentQueryService`, sin consultas SQL de un módulo a las tablas de otro.	Aceptado	Aplica el principio de inversión de dependencias y protege los límites entre módulos (RNF-05). Se renombran `IConsultaAcademica` e `IConsultaMatriculas`.
3.3	Dependencias: Matrícula → Gestión académica; Calificaciones → Gestión académica y Matrícula.	Aceptado	No hay ciclos entre módulos. Se verifica en el diagrama propio.
3.4	**"API Gateway"** como capa de entrada.	Modificado	En un monolito no hace falta un producto de API Gateway aparte. La seguridad (JWT, roles, límite de peticiones, manejo de errores) es un *pipeline* de middleware dentro de la misma aplicación, y el HTTPS termina en el balanceador (vista de despliegue).
3.5	Auditoría por un **bus de eventos en memoria asíncrono**.	Modificado	Error de diseño. Si el proceso se cae después de guardar la nota pero antes de procesar el evento, el cambio queda sin auditar, lo que viola RF-12. Además, la respuesta se contradice: en un punto dice "asíncrona" y en otro "sincrónicamente". Se reemplaza por la interfaz `IRegistroAuditoria`, que el módulo llama **dentro de la misma transacción** del cambio (ADR-005).
3.6	La matrícula abre la transacción en el esquema `enrollment`, pero bloquea la **fila del grupo**, que vive en el esquema académico.	Modificado	Inconsistencia: Matrícula estaría escribiendo en tablas de Gestión académica y rompería el aislamiento que la misma IA defiende en 3.2. Se resuelve con lo que la IA ya insinuaba en su esquema (`Quotas`): Matrícula es dueña de la tabla `cupo_grupo` (cupo máximo y ocupados) y el bloqueo se hace sobre esa fila (ADR-002).
3.7	Cerrar sesión con **lista negra de tokens en Redis**.	Modificado	Redis se descartó en P2. Se usa un token de acceso de vida corta (15 min) más un token de renovación guardado en el esquema de identidad, que se revoca al cerrar sesión (ADR-004).
3.8	Endpoints REST versionados `/api/v1/...` por módulo.	Aceptado	Cumple RNF-08. Se documentarán con OpenAPI.
3.9	Faltan endpoints para **publicar** calificaciones y **cerrar** el periodo de notas.	Modificado (agregado)	Sin ellos no se cumplen RF-09 (el estudiante solo ve notas publicadas) ni RN-12 (cierre). Se agregan `POST /calificaciones/grupos/{id}/publicar` y `POST /academico/periodos/{id}/cerrar-notas`.
3.10	Prerrequisitos de asignaturas y datos de *tenant*.	Descartado	Ningún RF pide prerrequisitos ni multiinstitución. Se dejan como evolución futura.
3.11	Reportes con lectura directa de varias tablas mediante vistas SQL (CQRS ligero) y **réplicas de lectura**.	Modificado	Se aceptan las vistas de solo lectura con un usuario de base de datos sin permisos de escritura. La réplica de lectura solo se agrega si la prueba de carga muestra que los reportes afectan a RNF-02.
3.12	Búsqueda global `GET /search?q=` en el módulo de reportes.	Modificado	RF-11 pide buscar "según el módulo". La búsqueda va en los listados de cada módulo (`?q=&periodo=&programa=&estado=`) y aplica los filtros del rol (RNF-11).
3.13	Exportar reportes a PDF y Excel.	Modificado	RF-10 pide reportes básicos. Exportar a CSV queda como "debería" y a PDF como "podría".
3.14	Justificación obligatoria al modificar una nota, guardada en la auditoría.	Aceptado	Refuerza RN-12 y RF-12.
3.15	Diagrama Mermaid generado.	Modificado	El código es válido, pero el diagrama resultante es **ilegible** (ver `P3-diagrama-ia.png`): muchas líneas cruzadas, el bus de eventos aparece como un recuadro vacío y mezcla nombres en inglés y español. El modelo final se redibuja en dos diagramas más simples (módulos y estructura interna de un módulo).

Balance P3: 5 aceptadas, 9 modificadas, 1 descartada. Fue la respuesta con más correcciones. La IA describió bien los módulos y sus interfaces, pero su propuesta de auditoría asíncrona y el bloqueo sobre tablas de otro módulo contradecían requisitos y principios que ella misma había defendido en P2.

P4 · modelo de datos conceptual

#	Propuesta de la IA	Decisión	Justificación
4.1	Las diez entidades del enunciado más `Horario`, `UsuarioRol` y `Auditoria`, con atributos y claves.	Aceptado	Cubre las entidades exigidas en 6.4. El diagrama ER generado es legible (`P4-diagrama-ia.png`).
4.2	Relación **M:N entre Usuario y Rol** (`UsuarioRol`).	Aceptado	Mejor que nuestro supuesto S-05 (un rol por usuario): resuelve con naturalidad al docente que además coordina. Se actualiza S-05.
4.3	Estudiante y Docente como extensión 1:0..1 de Usuario.	Aceptado	Separa las credenciales (Identidad) de los datos académicos (Gestión académica).
4.4	Matrícula en **cabecera y detalle** (`Matricula` por periodo y `DetalleMatricula` por grupo).	Aceptado	Representa bien "matricular en asignaturas o grupos de un periodo" (RF-07).
4.5	`id_asignatura` **desnormalizado** en `DetalleMatricula` para la restricción única.	Modificado	Sin control, un detalle podría quedar con un grupo de una asignatura y un `id_asignatura` de otra. Se agrega una clave foránea compuesta `(id_grupo, id_asignatura)` → `Grupo(id_grupo, id_asignatura)` que obliga a que coincidan.
4.6	`cupo_ocupado` dentro de **Grupo** y bloqueo `FOR UPDATE` sobre Grupo.	Modificado	Repite el error del P3: Matrícula escribiría en tablas de Gestión académica. Grupo conserva `cupo_maximo` y se agrega la entidad **CupoGrupo** (1:1 con Grupo, del módulo Matrículas) con `ocupados` (ADR-002).
4.7	`porcentaje` del corte guardado en **cada calificación**.	Modificado	Error de normalización: el 30 % del "Parcial 1" se repetiría en la nota de cada estudiante y podría quedar distinto entre ellos. Se crea la entidad **Evaluacion** (corte del grupo con nombre y porcentaje); `Calificacion` queda como la nota de un estudiante en una evaluación.
4.8	Calificación **sin estado de publicación**.	Modificado (agregado)	RF-09 exige que el estudiante vea solo notas publicadas. Se agrega `estado` (Borrador o Publicada) en Calificación.
4.9	Asignatura cuelga directamente de Programa; **no hay plan de estudios**.	Modificado (agregado)	RF-05 pide gestionar planes de estudio. Se agrega **PlanEstudio** (Programa 1:N PlanEstudio 1:N Asignatura).
4.10	PeriodoAcademico solo con fecha de inicio, fecha de fin y estado.	Modificado	Faltan las fechas de matrícula y de registro de notas, y el cierre (RN-11, RN-12). Se agregan `inicio_matricula`, `fin_matricula`, `limite_notas` y `notas_cerradas`.
4.11	Restricciones únicas `(id_estudiante, id_periodo)`, `(id_matricula, id_asignatura)`, `(id_matricula, id_grupo)` y `CHECK (ocupados <= cupo_maximo)`.	Aceptado	Son la segunda barrera de ADR-002 en la base de datos. El CHECK pasa a la entidad CupoGrupo.
4.12	Cruce de horarios con `EXCLUDE USING GIST`.	Descartado	Esa restricción compara filas de una misma tabla; el cruce de un estudiante involucra Horario, Grupo y DetalleMatricula. Se valida en el servicio de dominio dentro de la transacción (RN-13).
4.13	Auditoría con valores anteriores y nuevos en JSONB.	Aceptado	Cumple RF-12 y ADR-005. Se descarta otra vez la etiqueta WORM (ver 1.14).
4.14	Falta la entidad para los **tokens de renovación**.	Modificado (agregado)	ADR-004 la necesita para poder cerrar sesión. Se agrega **Sesion** en el esquema de identidad.

Balance P4: 6 aceptadas, 7 modificadas, 1 descartada. El modelo de la IA es sólido como base, pero le faltaban tres elementos que el enunciado pide (planes de estudio, notas publicadas y fechas del periodo) y tenía un problema de normalización en los porcentajes.

P5 · secuencia de la matrícula

#	Propuesta de la IA	Decisión	Justificación
5.1	Escenario de dos solicitudes simultáneas por el último cupo: Tx1 bloquea, Tx2 espera, Tx1 confirma, Tx2 lee 30/30, revierte y recibe **409 Conflict**.	Aceptado	Es exactamente el comportamiento que exige RNF-07 y el escenario de prueba de ADR-002.
5.2	**Ordenar los `id_grupo` de forma ascendente** antes de bloquear cuando se matriculan varios grupos a la vez.	Aceptado	Aporte valioso que el equipo no había considerado: evita interbloqueos (*deadlocks*) entre dos matrículas con grupos en común. Se agrega a ADR-002.
5.3	Bloqueo sobre la tabla **Grupo**.	Modificado	Mismo error que en P3 y P4. El bloqueo va sobre CupoGrupo (ADR-002).
5.4	Valida el estado del estudiante y los duplicados **después** de tomar el bloqueo.	Modificado	Así el bloqueo se retiene más tiempo del necesario. Las validaciones que no dependen del cupo (estudiante activo, periodo abierto, cruce de horario) se hacen **antes** del bloqueo. Dentro del bloqueo solo quedan el cupo, el duplicado y la escritura.
5.5	Auditoría **asíncrona** después del COMMIT "para no frenar la respuesta".	Modificado	Tercera vez que la IA lo propone. Se mantiene ADR-005: la auditoría va dentro de la transacción. Una inserción adicional cuesta milisegundos y garantiza que ninguna matrícula quede sin registro.
5.6	Bloqueo retenido "≤ 15 ms".	Modificado	Es una cifra no verificada. Se trata como hipótesis y se mide en la prueba de concurrencia de ADR-002.
5.7	No define qué pasa si la espera del bloqueo se alarga.	Modificado (agregado)	Se agrega un tiempo máximo de espera (`lock_timeout` de 5 s) con respuesta "intente de nuevo" (503), para no agotar las conexiones del pool.
5.8	El actor que matricula es el **estudiante**.	Modificado	Según el supuesto S-02, en esta versión matricula el coordinador. El flujo es el mismo; solo cambia el rol autorizado.
5.9	Participante "API Gateway / Auth".	Modificado	Se reemplaza por el pipeline de seguridad dentro del monolito (ver 3.4).

Balance P5: 2 aceptadas, 7 modificadas. La IA resolvió bien el problema central de concurrencia y aportó la regla de ordenar los bloqueos. Pero insistió en la auditoría asíncrona y en bloquear la tabla de otro módulo, aunque ya se habían corregido.

P6 · despliegue, seguridad y respaldo

#	Propuesta de la IA	Decisión	Justificación
6.1	**Cloudflare** (plan gratuito) delante del sistema: TLS, mitigación de DDoS, WAF básico y CDN.	Aceptado	Cubre RNF-01 y la CDN que quedó pendiente en P2 sin costo adicional. También permite el límite de peticiones (*rate limiting*) en el borde.
6.2	Interfaz web compilada en **almacenamiento de objetos + CDN**.	Aceptado	Los servidores de la API no gastan CPU en archivos estáticos.
6.3	Balanceador (Nginx o el del proveedor) con *health checks* y **2 instancias** de la API (2 vCPU, 4 GB) en red privada.	Aceptado	Coincide con ADR-001 y ADR-004. El tamaño de las máquinas es una estimación que se confirma con la prueba de carga (RNF-02).
6.4	**Redis** para lista negra de tokens, límite de peticiones y caché.	Descartado	Cuarta vez que la IA lo propone. Ya se resolvió sin Redis: los tokens en ADR-004, el límite de peticiones en Cloudflare y la caché de catálogos en memoria (P2). Agregarlo sumaría un componente y un punto de falla sin necesidad.
6.5	**PostgreSQL administrado con réplica en espera** y conmutación automática (60 a 120 s).	Aceptado	Elimina el punto único de falla que ADR-001 había dejado abierto. Se exige que la réplica esté en otra zona de disponibilidad. Se documenta en ADR-003.
6.6	Base de datos en **subred privada** sin IP pública; el puerto 5432 solo acepta tráfico de los nodos de la API; conexiones con TLS.	Aceptado	Defensa en profundidad para RNF-01 y RNF-11.
6.7	Secretos (llaves JWT, credenciales) fuera del repositorio, inyectados por un **gestor de secretos**.	Aceptado	Cumple además la indicación del enunciado de no publicar credenciales en el repositorio.
6.8	Token de renovación de **7 días** con rotación en cada uso.	Modificado	Se acepta la rotación (mejora ADR-004). Se mantiene una vigencia de 8 horas: en una institución educativa muchos usuarios entran desde computadores compartidos (salas, bibliotecas), y 7 días es demasiado.
6.9	**RPO ≤ 15 min** y **RTO ≤ 2 h** con respaldo diario completo más archivado continuo del WAL (recuperación a un punto en el tiempo).	Aceptado	Mejora nuestra propuesta inicial (RPO 24 h, RTO 4 h) con bajo costo, porque los servicios administrados ya incluyen PITR. Se actualiza RNF-09 en el análisis.
6.10	Respaldos cifrados (AES-256) en almacenamiento externo, regla 3-2-1, **Object Lock** con retención de 30 días.	Aceptado	Protege contra borrado accidental y *ransomware*. A diferencia de la auditoría (1.14), aquí el bloqueo de objetos es una función barata del almacenamiento y sí se justifica.
6.11	Restauración con un script probado **cada trimestre** en un entorno de pruebas.	Aceptado	Es la evidencia de cumplimiento de RNF-09 en la matriz de trazabilidad.
6.12	Despliegues **rolling update** sin interrupción.	Aceptado	Evita que las actualizaciones consuman el margen de 3,6 h de RNF-03.
6.13	No menciona **monitoreo ni alertas**.	Modificado (agregado)	Sin monitoreo no se puede medir el 99,5 % ni reaccionar a tiempo. Se agrega un monitor externo de disponibilidad con alertas, y registros centralizados con el identificador de correlación.
6.14	No considera que Cloudflare y el proveedor de nube **procesan datos personales** de estudiantes.	Modificado (agregado)	En Colombia aplica la Ley 1581 de 2012 (habeas data). Se registra como riesgo que requiere validar los contratos de tratamiento de datos con los proveedores.
6.15	Diagrama de despliegue generado.	Modificado	Otra vez ilegible (`P6-diagrama-ia.png`): textos que no se leen y flechas sueltas. Se redibuja.

Balance P6: 10 aceptadas, 4 modificadas, 1 descartada. Fue la respuesta más útil de las seis: elevó el nivel de la propuesta en disponibilidad y respaldo.

P7 · revisión crítica de la propia propuesta

La IA criticó **su propia** propuesta de esta conversación. Varias de sus críticas más graves apuntan a elementos que el equipo ya había corregido en las evaluaciones anteriores. Eso confirma esas correcciones.

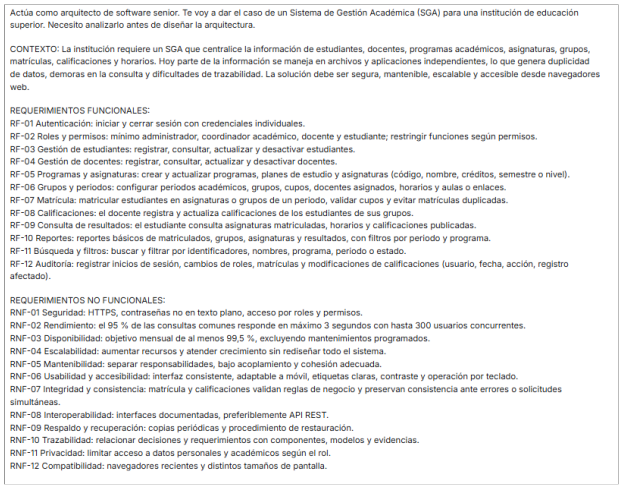
#	Crítica de la IA	Decisión	Justificación
7.1	**Redis único** es punto único de falla (límite de peticiones, lista negra, caché).	Ya resuelto	Redis se descartó en P2, P3 y P6. El diseño final no lo usa.
7.2	La auditoría por **bus de eventos en memoria** se pierde si el proceso cae después del COMMIT; sugiere el patrón *Outbox*.	Ya resuelto	ADR-005 escribe la auditoría en la misma transacción del cambio. Es más simple que *Outbox* y no deja eventos pendientes. La prueba de caos que propone la IA (`kill -9` durante matrículas) se adopta para **demostrar** que no hay discrepancias.
7.3	Bloqueo sobre la **tabla Grupo**.	Ya resuelto	ADR-002 bloquea `cupo_grupo`, que pertenece al módulo Matrículas.
7.4	**Balanceador único** como punto único de falla.	Aceptado	Crítica válida. Se usa el balanceador administrado del proveedor, que es redundante por diseño, en lugar de un Nginx propio en una sola máquina. Se actualiza ADR-003.

#	Crítica de la IA	Decisión	Justificación
7.5	**Agotamiento del pool de conexiones** de PostgreSQL con dos nodos.	Aceptado	Se fija un pool máximo por nodo (20 conexiones) y se usa el *pooler* que ofrece el servicio administrado (tipo PgBouncer). Se actualiza ADR-003.
7.6	**Contención extrema** si 250 usuarios piden el mismo grupo al mismo tiempo: la cola de bloqueos supera los 3 s.	Modificado	El riesgo es real si se habilita la automatrícula. Hoy matricula el coordinador (S-02), lo que reduce mucho la concurrencia. Se agrega a ADR-002 una **lectura previa sin bloqueo**: si el grupo ya está lleno, se rechaza de inmediato sin hacer cola. Se valida con la prueba de 300 usuarios sobre 5 cupos que propone la IA.
7.7	Los **reportes pesados** degradan el pool transaccional en el cierre de notas.	Aceptado	Los reportes usan un pool de conexiones separado y pequeño, con tiempo máximo por consulta. La réplica de lectura se activa si la prueba de carga lo exige (ver 3.11).
7.8	La **capacidad del aula** no se considera: un grupo de 40 podría quedar en un aula de 25.	Aceptado	Buena observación sobre RF-06. Se agrega la entidad **Aula** con su capacidad y la regla RN-15: en grupos presenciales, el cupo no puede superar la capacidad del aula.
7.9	Un **superusuario de la base de datos** puede alterar la auditoría.	Aceptado	Riesgo residual real. Se mitiga con una exportación diaria de los eventos de auditoría al almacenamiento con bloqueo de objetos (el mismo de los respaldos, ADR-003) y con el registro de actividad del propio servicio administrado. Así, una alteración posterior se puede detectar comparando.
7.10	**Usuarios con varios roles** sin un contexto de sesión activo.	Aceptado	Crítica válida que surge de aceptar el M:N en P4. El usuario elige el perfil con el que entra y el token lleva un único **rol activo**. Los permisos se evalúan con ese rol. Se actualiza ADR-004.
7.11	RNF-06 sin verificación automática.	Aceptado	Se incluyen axe-core y Lighthouse CI en el pipeline de integración continua (matriz de trazabilidad, RNF-06).
7.12	RNF-10 sin estándar de ADR ni versionado de la API.	Modificado	La IA no conoce los ADR del equipo (ADR-001 a ADR-005 ya existen). Se acepta lo que sí faltaba: versionado semántico de la especificación OpenAPI y commits que citan el requerimiento (por ejemplo, `RF-07: ...`).
7.13	Faltan **prerrequisitos** de asignaturas (RF-05).	Descartado	RF-05 no los pide (pide código, nombre, créditos y semestre). Ya se había descartado en 3.10. Queda como evolución futura.
7.14	Calificación **a ciegas** (*blind grading*).	Descartado	No lo pide ningún requerimiento, y el docente necesita saber a quién califica. La privacidad de las notas ya la cubre RNF-11 con filtros por rol y propietario.
7.15	Plan de validación: prueba de concurrencia extrema, prueba de caos, simulacro de desastre y prueba de carga con mezcla 70 % lecturas, 20 % matrículas y 10 % reportes, de 50 a 500 usuarios.	Aceptado	Plan concreto y medible. Se incorpora al análisis de riesgos y a la matriz de trazabilidad.

Balance P7: 8 aceptadas, 2 modificadas, 2 descartadas y 3 que ya estaban resueltas.

Anexo B. Capturas de la conversación en AI Studio

Capturas en el orden de la conversación. La primera de cada prompt muestra lo que envié.



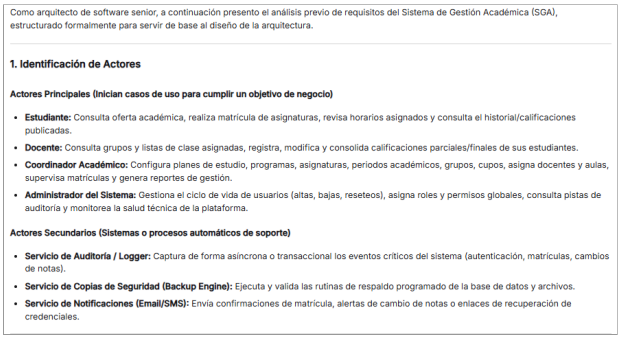
Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión Académica (SGA) para una institución de educación superior. Necesito analizarlo antes de diseñar la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes, programas académicos, asignaturas, grupos, matrículas, calificaciones y horarios. Hoy parte de la información se maneja en archivos y aplicaciones independientes, lo que genera duplicidad de datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura, mantenible, escalable y accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante; restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas (código, nombre, créditos, semestre o nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados, horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones (usuario, fecha, acción, registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300 usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras, contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y preservan consistencia ante errores o solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

P1-01 · análisis del problema



Como arquitecto de software senior, a continuación presento el análisis previo de requisitos del Sistema de Gestión Académica (SGA), estructurado formalmente para servir de base al diseño de la arquitectura.

1. Identificación de Actores

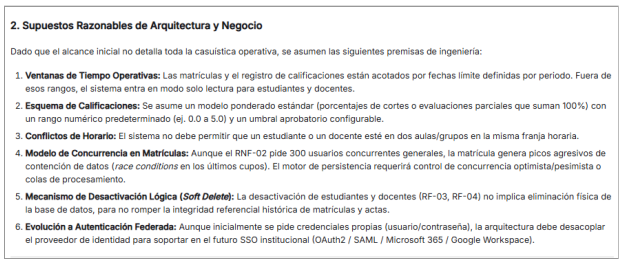
Actores Principales (Inician casos de uso para cumplir un objetivo de negocio)

- **Estudiante:** Consulta oferta académica, realiza matrícula de asignaturas, revisa horarios asignados y consulta el historial/calificaciones publicadas.
- **Docente:** Consulta grupos y listas de clase asignadas, registra, modifica y consolida calificaciones parciales/finales de sus estudiantes.
- **Coordinador Académico:** Configura planes de estudio, programas, asignaturas, periodos académicos, grupos, cupos, asigna docentes y aulas, supervisa matrículas y genera reportes de gestión.
- **Administrador del Sistema:** Gestiona el ciclo de vida de usuarios (altas, bajas, reseteos), asigna roles y permisos globales, consulta pistas de auditoría y monitorea la salud técnica de la plataforma.

Actores Secundarios (Sistemas o procesos automáticos de soporte)

- **Servicio de Auditoría / Logger:** Captura de forma asíncrona o transaccional los eventos críticos del sistema (autenticación, matrículas, cambios de notas).
- **Servicio de Copias de Seguridad (Backup Engine):** Ejecuta y valida las rutinas de respaldo programado de la base de datos y archivos.
- **Servicio de Notificaciones (Email/SMS):** Envía confirmaciones de matrícula, alertas de cambio de notas o enlaces de recuperación de credenciales.

P1-02 · análisis del problema

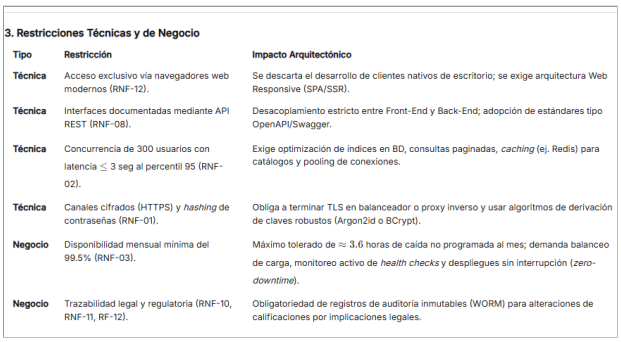


2. Supuestos Razonables de Arquitectura y Negocio

Dado que el alcance inicial no detalla toda la casuística operativa, se asumen las siguientes premisas de ingeniería:

1. **Ventanas de Tiempo Operativas:** Las matrículas y el registro de calificaciones están acotados por fechas límite definidas por periodo. Fuera de esos rangos, el sistema entra en modo solo lectura para estudiantes y docentes.
2. **Esquema de Calificaciones:** Se asume un modelo ponderado estándar (porcentajes de cortes o evaluaciones parciales que suman 100%) con un rango numérico predeterminado (ej. 0.0 a 5.0) y un umbral aprobatorio configurable.
3. **Conflictos de Horario:** El sistema no debe permitir que un estudiante o un docente esté en dos aulas/grupos en la misma franja horaria.
4. **Modelo de Concurrencia en Matrículas:** Aunque el RNF-02 pide 300 usuarios concurrentes generales, la matrícula genera picos agresivos de contención de datos (*race conditions* en los últimos cupos). El motor de persistencia requerirá control de concurrencia optimista/pesimista o colas de procesamiento.
5. **Mecanismo de Desactivación Lógica (*Soft Delete*):** La desactivación de estudiantes y docentes (RF-03, RF-04) no implica eliminación física de la base de datos, para no romper la integridad referencial histórica de matrículas y actas.
6. **Evolución a Autenticación Federada:** Aunque inicialmente se pide credenciales propias (usuario/contraseña), la arquitectura debe desacoplar el proveedor de identidad para soportar en el futuro SSO institucional (OAuth2 / SAML / Microsoft 365 / Google Workspace).

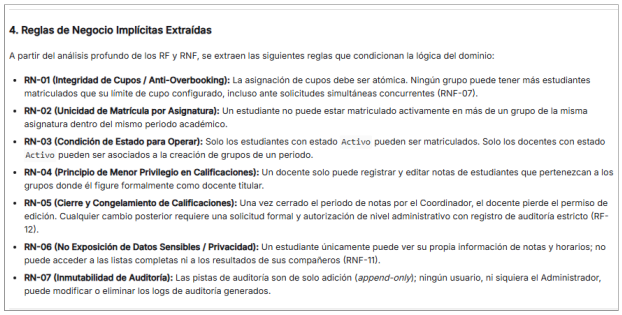
P1-03 · análisis del problema



3. Restricciones Técnicas y de Negocio

Tipo	Restricción	Impacto Arquitectónico
Técnica	Acceso exclusivo vía navegadores web modernos (RNF-12).	Se descarta el desarrollo de clientes nativos de escritorio; se exige arquitectura Web Responsive (SPA/SSR).
Técnica	Interfaces documentadas mediante API REST (RNF-08).	Desacoplamiento estricto entre Front-End y Back-End; adopción de estándares tipo OpenAPI/Swagger.
Técnica	Concurrencia de 300 usuarios con latencia $\leq$ 3 seg al percentil 95 (RNF-02).	Exige optimización de índices en BD, consultas paginadas, *caching* (ej. Redis) para catálogos y pooling de conexiones.
Técnica	Canales cifrados (HTTPS) y *hashing* de contraseñas (RNF-01).	Obliga a terminar TLS en balanceador o proxy inverso y usar algoritmos de derivación de claves robustos (Argon2id o BCrypt).
Negocio	Disponibilidad mensual mínima del 99.5% (RNF-03).	Máximo tolerado de ≈ 3.6 horas de caída no programada al mes; demanda balanceo de carga, monitoreo activo de *health checks* y despliegues sin interrupción (*zero-downtime*).
Negocio	Trazabilidad legal y regulatoria (RNF-10, RNF-11, RF-12).	Obligatoriedad de registros de auditoría inmutables (WORM) para alteraciones de calificaciones por implicaciones legales.

P1-04 · análisis del problema

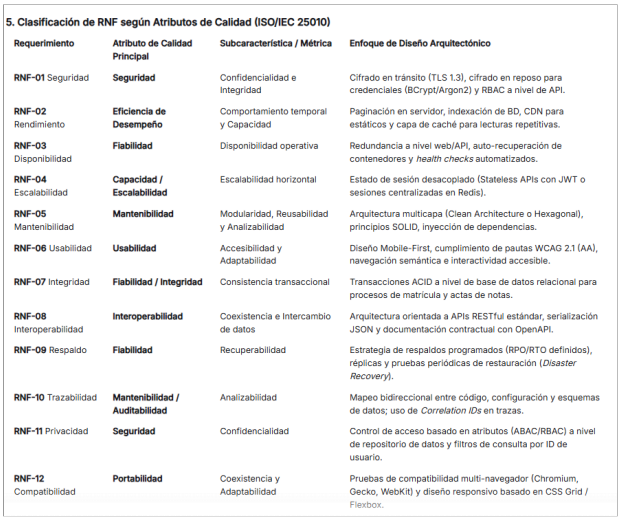


4. Reglas de Negocio Implícitas Extraídas

A partir del análisis profundo de los RF y RNF, se extraen las siguientes reglas que condicionan la lógica del dominio:

- **RN-01 (Integridad de Cupos / Anti-Overbooking):** La asignación de cupos debe ser atómica. Ningún grupo puede tener más estudiantes matriculados que su límite de cupo configurado, incluso ante solicitudes simultáneas concurrentes (RNF-07).
- **RN-02 (Unicidad de Matrícula por Asignatura):** Un estudiante no puede estar matriculado activamente en más de un grupo de la misma asignatura dentro del mismo periodo académico.
- **RN-03 (Condición de Estado para Operar):** Solo los estudiantes con estado `Activo` pueden ser matriculados. Solo los docentes con estado `Activo` pueden ser asociados a la creación de grupos de un periodo.
- **RN-04 (Principio de Menor Privilegio en Calificaciones):** Un docente solo puede registrar y editar notas de estudiantes que pertenezcan a los grupos donde él figure formalmente como docente titular.
- **RN-05 (Cierre y Congelamiento de Calificaciones):** Una vez cerrado el periodo de notas por el Coordinador, el docente pierde el permiso de edición. Cualquier cambio posterior requiere una solicitud formal y autorización de nivel administrativo con registro de auditoría estricto (RF-12).
- **RN-06 (No Exposición de Datos Sensibles / Privacidad):** Un estudiante únicamente puede ver su propia información de notas y horarios; no puede acceder a las listas completas ni a los resultados de sus compañeros (RNF-11).
- **RN-07 (Inmutabilidad de Auditoría):** Las pistas de auditoría son de solo adición (*append-only*); ningún usuario, ni siquiera el Administrador, puede modificar o eliminar los logs de auditoría generados.

P1-05 · análisis del problema



5. Clasificación de RNF según Atributos de Calidad (ISO/IEC 25010)

Requerimiento	Atributo de Calidad Principal	Subcaracterística / Métrica	Enfoque de Diseño Arquitectónico
RNF-01 Seguridad	**Seguridad**	Confidencialidad e Integridad	Cifrado en tránsito (TLS 1.3), cifrado en reposo para credenciales (BCrypt/Argon2) y RBAC a nivel de API.
RNF-02 Rendimiento	**Eficiencia de Desempeño**	Comportamiento temporal y Capacidad	Paginación en servidor, indexación de BD, CDN para estáticos y capa de caché para lecturas repetitivas.
RNF-03 Disponibilidad	**Fiabilidad**	Disponibilidad operativa	Redundancia a nivel web/API, auto-recuperación de contenedores y *health checks* automatizados.
RNF-04 Escalabilidad	**Capacidad / Escalabilidad**	Escalabilidad horizontal	Estado de sesión desacoplado (Stateless APIs con JWT o sesiones centralizadas en Redis).
RNF-05 Mantenibilidad	**Mantenibilidad**	Modularidad, Reusabilidad y Analizabilidad	Arquitectura multicapa (Clean Architecture o Hexagonal), principios SOLID, inyección de dependencias.
RNF-06 Usabilidad	**Usabilidad**	Accesibilidad y Adaptabilidad	Diseño Mobile-First, cumplimiento de pautas WCAG 2.1 (AA), navegación semántica e interactividad accesible.
RNF-07 Integridad	**Fiabilidad / Integridad**	Consistencia transaccional	Transacciones ACID a nivel de base de datos relacional para procesos de matrícula y actas de notas.
RNF-08 Interoperabilidad	**Interoperabilidad**	Coexistencia e Intercambio de datos	Arquitectura orientada a APIs RESTful estándar, serialización JSON y documentación contractual con OpenAPI.
RNF-09 Respaldo	**Fiabilidad**	Recuperabilidad	Estrategia de respaldos programados (RPO/RTO definidos), réplicas y pruebas periódicas de restauración (*Disaster Recovery*).
RNF-10 Trazabilidad	**Mantenibilidad / Auditabilidad**	Analizabilidad	Mapeo bidireccional entre código, configuración y esquemas de datos; uso de *Correlation IDs* en trazas.
RNF-11 Privacidad	**Seguridad**	Confidencialidad	Control de acceso basado en atributos (ABAC/RBAC) a nivel de repositorio de datos y filtros de consulta por ID de usuario.
RNF-12 Compatibilidad	**Portabilidad**	Coexistencia y Adaptabilidad	Pruebas de compatibilidad multi-navegador (Chromium, Gecko, WebKit) y diseño responsivo basado en CSS Grid / Flexbox.

P1-06 · análisis del problema

Con base en el análisis anterior, propón el estilo arquitectónico para el SGA.
Compara al menos tres opciones: arquitectura en capas, monolito modular y microservicios.
Para cada una evalúa: complejidad de desarrollo y operación, rendimiento con 300 usuarios concurrentes, disponibilidad del 99,5 %, escalabilidad, mantenibilidad y consistencia en la matrícula ante solicitudes simultáneas.
Al final recomienda una opción y justifícala frente a los requerimientos no funcionales. Responde en español con una tabla comparativa.

P2-01 · estilo arquitectónico

1. Evaluación de los Estilos Arquitectónicos

A. Arquitectura en Capas Tradicional (N-Tier: Presentación → Lógica de Negocio → Acceso a Datos)

- **Complejidad:** Muy baja en desarrollo y operación. Curva de aprendizaje mínima para el equipo.
- **Rendimiento (300 concurrentes):** Cumple con el objetivo sin problemas si la base de datos relacional está adecuadamente indexada.
- **Disponibilidad (99,5 %):** Alcanzable mediante un balanceador de carga que distribuya tráfico entre 2 o más instancias de la aplicación.
- **Escalabilidad:** Escalamiento horizontal del bloque completo. Ineficiente si solo el módulo de matrícula sufre congestión.
- **Mantenibilidad: Riesgo crítico.** Con el paso del tiempo, las capas monolíticas tradicionales tienden al fenómeno de *"Gran Bola de Lodo"* (*Big Ball of Mud*). La lógica de matrícula, notas y auditoría suele acoplarse fuertemente en las mismas entidades y servicios compartidos.
- **Consistencia en Matrícula:** Alta. Permite transacciones ACID locales directas en una única base de datos.

B. Monolito Modular (Modular Monolith)

- **Complejidad:** Media en desarrollo (exige disciplina de diseño para delimitar contextos y módulos independientes con APIs internas estrictas) y baja en operación (se empaqueta y despliega como un único artefacto o contenedor).
- **Rendimiento (300 concurrentes): Óptimo.** La comunicación intermodular ocurre en memoria (*in-process method calls*), eliminando la latencia de red y la sobrecarga de serialización HTTP/JSON. Supera con holgura la meta de < 3 s.
- **Disponibilidad (99,5 %):** Muy alta. Dos réplicas activas detrás de un balanceador (ej. Nginx o AWS ALB) toleran caídas de nodo sin interrumpir el servicio.
- **Escalabilidad:** Escalamiento horizontal eficiente del artefacto. Si a futuro un módulo específico (ej. Matrícula) requiere independizarse, sus límites limpios facilitan su extracción sin reescribir el sistema.
- **Mantenibilidad: Excelente.** Separa responsabilidades por dominio de negocio (Módulo Estudiantes, Módulo Calificaciones, Módulo Matrícula, Módulo Auditoría), garantizando alta cohesión y bajo acoplamiento sin la sobrecarga operativa de los microservicios.
- **Consistencia en Matrícula: Excelente.** Gestiona la contención de cupos y concurrencia mediante transacciones ACID nativas y aislamiento a nivel de base de datos relacional (bloqueo pesimista o selectivo por fila).

C. Microservicios (Servicios distribuidos independientes con BD por servicio)

- **Complejidad: Extremadamente alta.** Requiere infraestructura avanzada (orquestador tipo Kubernetes, *service mesh*, *distributed tracing*, pasarela de API) y gestión compleja de despliegues independientes. Sobrediseño (*over-engineering*) para el tamaño de la institución.
- **Rendimiento (300 concurrentes):** Puede verse penalizado por saltos de red (*network hops*) y latencia de serialización en consultas compuestas (ej. generar el consolidado de matrícula y notas de un estudiante).
- **Disponibilidad (99,5 %):** Teóricamente alta por aislamiento de fallas, pero en la práctica vulnerable a fallas en cascada y dependencia de la red si no se implementan patrones complejos de resiliencia (*circuit breaker*, reintentos).
- **Escalabilidad:** Máxima a nivel granular, permitiendo asignar recursos únicamente al servicio de matrículas.
- **Mantenibilidad:** Alta a nivel de código de cada servicio, pero muy difícil de trazar y mantener en cambios que cruzan contratos de múltiples servicios.
- **Consistencia en Matrícula: Crítica y de alto riesgo.** Al requerir bases de datos separadas, la validación de cupos y cruces de horario obligaría a implementar consistencia eventual y patrones Saga. Esto incrementa dramáticamente el riesgo de *race conditions* y sobrecupo (*overbooking*) en periodos pico de matrícula.

P2-02 · estilo arquitectónico

2. Tabla Comparativa

Criterio de Evaluación	Arquitectura en Capas	Monolito Modular	Microservicios
Complejidad de Desarrollo	**Baja** (un solo proyecto, patrones simples).	**Media** (diseño DDD, contratos internos).	**Muy Alta** (manejo de sistemas distribuidos).
Complejidad Operativa (DevOps)	**Baja** (un único artefacto a desplegar).	**Baja** (un único artefacto o contenedor).	**Extremadamente Alta** (K8s, redes, observabilidad distribuida).
Rendimiento (300 conc. / ≤ 3 s)	**Bueno** (suficiente para la carga).	**Excelente** (llamadas en memoria, sin latencia de red).	**Moderado/Variable** (latencia de red inter-servicio).
Disponibilidad (99,5 %)	**Media-Alta** (2 nodos + Load Balancer).	**Alta** (2-3 réplicas + Load Balancer, rápido arranque).	**Alta** (aislamiento de fallos, pero depende de la red).
Escalabilidad (RNF-04)	**Media** (escala todo el bloque).	**Buena** (escala todo el bloque; desacoplado internamente).	**Excelente** (escala servicios específicos bajo demanda).
Mantenibilidad (RNF-05)	**Baja** (riesgo de acoplamiento espagueti).	**Alta** (módulos cohesivos y aislados por dominio).	**Media-Alta** (servicios pequeños, pero contratos complejos).
Consistencia en Matrícula (RNF-07)	**Alta** (transacciones ACID directas).	**Óptima** (ACID relacional, sin complejidad distribuida).	**Deficiente/Compleja** (requiere Sagas y consistencia eventual).

P2-03 · estilo arquitectónico

3. Recomendación Arquitectónica y Justificación

Estilo Recomendado: Monolito Modular (Modular Monolith)

Se recomienda adoptar una arquitectura de **Monolito Modular con persistencia relacional única y partición lógica de esquemas**, complementada con una interfaz web desacoplada (SPA o SSR) consumiendo una API REST.

Justificación frente a los Requerimientos No Funcionales:

1. **Integridad y Consistencia (RNF-07):** El proceso de matrícula con validación de cupos concurrentes y prevención de duplicados es el mayor riesgo del SGA. El monolito modular permite ejecutar transacciones atómicas (**ACID**) en la base de datos relacional (ej. PostgreSQL con `SELECT ... FOR UPDATE` o control de concurrencia optimista), garantizando que jamás se exceda un cupo ni se matricule una asignatura duplicada, eliminando la fragilidad de la consistencia eventual de los microservicios.
2. **Rendimiento y Concurrencia (RNF-02):** Un pico de 300 usuarios concurrentes es una carga ligera para la capacidad de cómputo moderna. Al mantener los módulos (Estudiantes, Asignaturas, Matrícula, Calificaciones) dentro del mismo proceso de ejecución, las llamadas entre módulos son instantáneas en memoria, garantizando holgadamente respuestas en menos de 3 segundos para el 95 % de las peticiones.
3. **Mantenibilidad y Modularidad (RNF-05):** A diferencia de la arquitectura en capas genérica, el monolito modular define barreras estrictas de dominio (Contextos Delimitados de DDD). El código de calificaciones no puede tocar directamente las tablas o entidades de matrícula, sino solo a través de interfaces públicas bien definidas, facilitando refactorizaciones, pruebas unitarias y reduciendo la deuda técnica.
4. **Escalabilidad y Crecimiento Futuro (RNF-04):** Permite atender el crecimiento institucional añadiendo réplicas idénticas detrás de un balanceador de carga. Además, si dentro de varios años el módulo de matrícula o el de autenticación requiere escalar independientemente, el aislamiento de sus interfaces permite extraerlo hacia un microservicio autónomo con un costo de refactorización mínimo.
5. **Disponibilidad y Recuperabilidad (RNF-03, RNF-09):** Desplegar 2 o 3 réplicas del monolito en contenedores (ej. Docker) detrás de un proxy/balanceador garantiza el 99,5 % de disponibilidad. La gestión de copias de seguridad y procedimientos de restauración (RTO/RPO) sobre una única base de datos relacional es directa y confiable, sin lidiar con restauraciones desincronizadas entre múltiples almacenes de datos.
6. **Seguridad y Auditoría Centralizada (RNF-01, RNF-10, RNF-11):** La aplicación de políticas de roles y permisos (RBAC) y el registro transaccional de auditoría (RF-12) se implementan mediante interceptores (*middleware*) centralizados y atómicos en el ciclo de vida de la petición, garantizando que ninguna mutación de notas quede sin registro auditable.

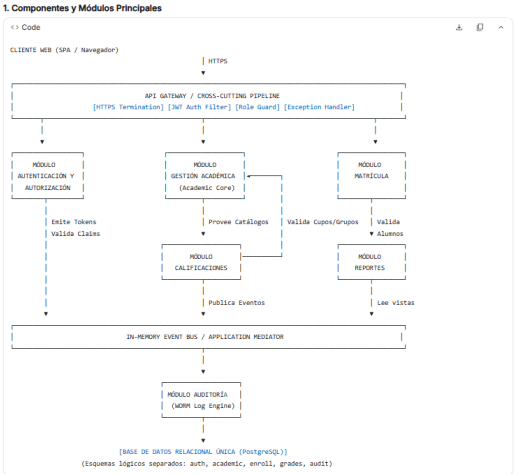
P2-04 · estilo arquitectónico

Para la arquitectura que recomendaste, describe la vista lógica:
los módulos o componentes principales, la responsabilidad de cada uno, las interfaces que exponen (por ejemplo, endpoints REST principales) y las dependencias entre ellos.
Debe quedar claro cómo se organizan autenticación y autorización, gestión académica, matrículas, calificaciones, reportes y auditoría.
Genera además el diagrama en sintaxis Mermaid.

P3-01 · componentes

El diseño se estructura bajo principios de *Domain-Driven Design* (DDD) mediante **Contextos Delimitados (*Bounded Contexts*)**, donde cada módulo es un paquete autónomo en el mismo proceso de despliegue, comunicándose hacia el exterior mediante APIs REST y entre sí mediante contratos/interfaces en memoria o eventos de dominio.

1. Componentes y Módulos Principales



P3-02 · componentes

2. Detalle de Módulos: Responsabilidades, Interfaces y Dependencias

2.1. Módulo de Autenticación y Autorización (IAM)

- **Responsabilidad:** Gestión del ciclo de vida de credenciales (hashing Argon2id), emisión y verificación de JSON Web Tokens (JWT), resolución de roles (`ADMINISTRADOR`, `COORDINADOR`, `DOCENTE`, `ESTUDIANTE`) y permisos.
- **Endpoints REST:**
  - `POST /api/v1/auth/login` (Retorna JWT firmado + *refresh token*).
  - `POST /api/v1/auth/logout` (Invalida el token en lista negra / Redis).
  - `PUT /api/v1/auth/users/{id}/roles` (Solo `ADMINISTRADOR`: asignación de perfiles).
- **Interfaz Interna (In-Memory Service):**
  - `ISecurityContext`: Expone método `GetCurrentUser(): UserContext` con `userId`, `roles`, y `tenant/metadata`.
- **Dependencias:** Ninguna (módulo base).

2.2. Módulo de Gestión Académica (Academic Core)

- **Responsabilidad:** Administración de entidades fundamentales: Estudiantes, Docentes, Programas, Asignaturas, Periodos, Aulas y Grupos con sus horarios y límites de cupo (RF-03, RF-04, RF-05, RF-06).
- **Endpoints REST:**
  - `POST|GET|PUT /api/v1/academic/programs` y `/courses` (Gestión de oferta).
  - `POST|GET|PUT /api/v1/academic/terms` (Configuración de períodos académicos).
  - `POST|GET|PUT /api/v1/academic/groups` (Creación de grupos, asignación docente y definición de cupos).
  - `POST|PUT /api/v1/academic/students` y `/professors` (Altas, bajas lógicas y modificaciones).
- **Interfaz Interna (In-Memory Service):**
  - `IAcademicQueryService`: Permite a otros módulos verificar si un estudiante está `Activo`, si un docente está asignado a un grupo específico o consultar los prerrequisitos de una asignatura.
- **Dependencias:** `ISecurityContext` (para validar roles administrativos/coordinadores).

P3-03 · componentes

2.3. Módulo de Matrícula (Enrollment Engine)

- **Responsabilidad:** Procesar la inscripción de asignaturas asegurando **consistencia transaccional estricta (ACID)**. Valida cupos en tiempo real, verifica que no existan cruces de horario, impide duplicidad de asignaturas y actualiza la ocupación del grupo (RF-07, RNF-07).
- **Endpoints REST:**
  - `POST /api/v1/enrollments` (Ejecuta la matrícula de uno o varios grupos).
  - `DELETE /api/v1/enrollments/{enrollmentId}/groups/{groupId}` (Cancelación de grupo).
  - `GET /api/v1/enrollments/my-schedule` (Horario del estudiante autenticado).
- **Interfaz Interna:**
  - `IEnrollmentQueryService`: Expone `GetEnrolledStudentsByGroup(groupId): List<StudentRef>` (usado por el Módulo de Calificaciones).
- **Dependencias:**
  - `IAcademicQueryService` (para validar estado del estudiante, validez del grupo y cupos máximos).
  - `IEventPublisher` (publica `StudentEnrolledEvent` hacia Auditoría).

2.4. Módulo de Calificaciones (Grades Management)

- **Responsabilidad:** Registro, edición y cálculo ponderado de notas parciales y finales. Bloquea modificaciones si el docente no es titular o si la ventana temporal del período cerró (RF-08, RF-09).
- **Endpoints REST:**
  - `GET /api/v1/grades/groups/{groupId}` (Lista de notas; acceso para el docente titular o coordinador).
  - `PUT /api/v1/grades/groups/{groupId}/evaluations` (Carga/actualización de notas por corte).
  - `GET /api/v1/grades/my-grades` (Solo lectura para el estudiante sobre sus notas publicadas).
- **Dependencias:**
  - `IAcademicQueryService` (valida asignación del docente al grupo y estado del periodo).
  - `IEnrollmentQueryService` (obtiene la lista oficial de matriculados del grupo).
  - `IEventPublisher` (publica `GradeModifiedEvent` con nota previa, nota nueva, autor y justificación hacia Auditoría).

P3-04 · componentes

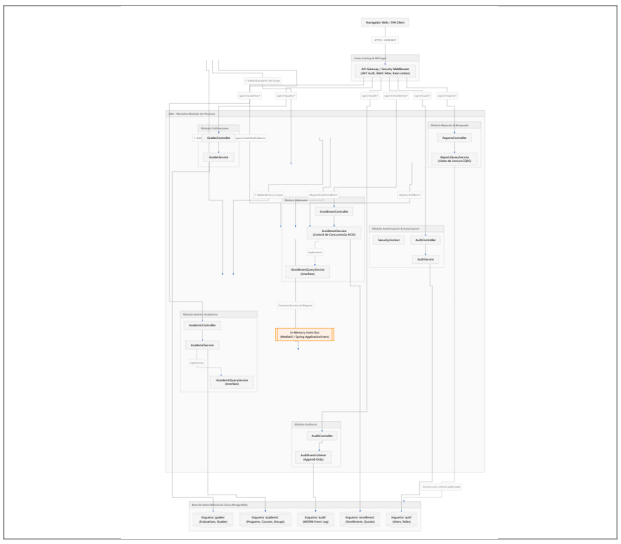
2.5. Módulo de Reportes y Búsqueda (Reporting & Search)

- **Responsabilidad:** Generación optimizada de consultas consolidadas, listas de clase, sábanas de notas y reportes institucionales en PDF/Excel sin degradar el procesamiento transaccional (RF-10, RF-11).
- **Endpoints REST:**
  - `GET /api/v1/reports/enrollments?termId=X&programId=Y`
  - `GET /api/v1/reports/grades-summary?termId=X&groupId=Y`
  - `GET /api/v1/search?q={term}&entity={student|teacher|group}`
- **Dependencias:** Consulta directamente las vistas de lectura en la base de datos relacional (patrón CQRS a nivel de base de datos usando vistas SQL optimizadas/réplicas de lectura).

2.6. Módulo de Auditoría (Audit Engine)

- **Responsabilidad:** Registro inmutable (*Append-Only*) de eventos sensibles: inicios de sesión, cambios de roles, matrículas efectuadas y alteraciones de calificaciones (RF-12, RNF-10).
- **Endpoints REST:**
  - `GET /api/v1/audit/logs?entity=X&userId=Y&fromDate=Z` (Restringido exclusivamente al `ADMINISTRADOR`).
- **Mecanismo de Ingesta:**
  - Suscriptor desacoplado del `In-Memory Event Bus`. Escucha eventos asíncronos (`GradeModifiedEvent`, `UserLoggedInEvent`, `RoleChangedEvent`, `EnrollmentCompletedEvent`) y persiste de forma asíncrona pero confiable en la tabla `audit.logs`.
- **Dependencias:** Ninguna (módulo sumidero de eventos).

P3-05 · componentes



P3-06 · componentes

4. Patrones de Colaboración Clave

1. **Matrícula Concurrente Atómica (RNF-07):**
   Cuando `EnrollmentController` recibe una solicitud, abre una transacción ACID en el esquema `enrollment`. Mediante `IAcademicQueryService` confirma el estado del estudiante y consulta el cupo. Ejecuta un bloqueo pesimista (`SELECT ... FOR UPDATE`) sobre la fila del grupo para restar el cupo en el mismo commit. Si la transacción finaliza con éxito, emite un evento `StudentEnrolledEvent` al bus en memoria.
2. **Modificación de Notas con Trazabilidad Estricta (RF-08, RF-12):**
   `GradesService` recibe el cambio de nota. Valida con `IAcademicQueryService` que el usuario autenticado sea el docente titular asignado. Al actualizar la nota en el esquema `grades`, se genera sincrónicamente un evento con el payload `{estudianteId, grupoId, notaAnterior, notaNueva, docenteId, justificacion}`. El `AuditEventListener` procesa el evento y escribe en el esquema inmutable `audit`.
3. **Aislamiento de Módulos:**
   Ningún módulo ejecuta consultas SQL directas contra las tablas de otro módulo (salvo el módulo de *Reportes*, que tiene permisos de solo lectura optimizados para analítica). Cualquier validación cruzada entre *Matrículas* y *Calificaciones* se realiza estrictamente a través de las interfaces (`IAcademicQueryService`, `IEnrollmentQueryService`).

P3-07 · componentes

Propón el modelo de datos conceptual del SGA con las entidades: Usuario, Rol, Estudiante, Docente, Programa, Asignatura, PeriodoAcadémico, Grupo, Matrícula y Calificación, más las que consideres necesarias (por ejemplo, para horarios o auditoría).
Indica los atributos principales y las cardinalidades de cada relación, y explica las restricciones que evitan matrículas duplicadas.
Genera el diagrama entidad-relación en sintaxis Mermaid (erDiagram)

P4-01 · modelo de datos

1. Entidades Principales y Atributos

1. **Usuario:** Credenciales y estado global de acceso.
   - `id_usuario` (PK), `email` (UK), `password_hash`, `nombre`, `apellido`, `estado` (Activo/Inactivo), `fecha_creacion`.
2. **Rol:** Catálogo de permisos.
   - `id_rol` (PK), `codigo_rol` (UK: ADMIN, COORDINADOR, DOCENTE, ESTUDIANTE), `descripcion`.
3. **UsuarioRol:** Tabla asociativa para soportar asignación múltiple de roles (M:N).
   - `id_usuario` (PK, FK), `id_rol` (PK, FK).
4. **Estudiante:** Extensión de perfil académico del usuario.
   - `id_estudiante` (PK), `id_usuario` (FK, UK, 1:1), `codigo_estudiante` (UK), `id_programa` (FK), `estado_academico` (Regular, Egresado, Suspendido).
5. **Docente:** Extensión del perfil docente del usuario.
   - `id_docente` (PK), `id_usuario` (FK, UK, 1:1), `codigo_docente` (UK), `especialidad`, `tipo_contrato`.
6. **Programa:** Oferta académica institucional.
   - `id_programa` (PK), `codigo_programa` (UK), `nombre`, `nivel_academico` (Pregrado, Posgrado), `creditos_totales`.
7. **Asignatura:** Unidades curriculares asociadas a un plan de estudios.
   - `id_asignatura` (PK), `codigo_asignatura` (UK), `id_programa` (FK), `nombre`, `creditos`, `semestre_nivel`.
8. **PeriodoAcademico:** Ciclos lectivos regulares o vacacionales.
   - `id_periodo` (PK), `codigo_periodo` (UK: ej. "2025-1"), `fecha_inicio`, `fecha_fin`, `estado` (Planificacion, Abierto, Cerrado).
9. **Grupo:** Instancia concreta de una asignatura dictada en un periodo.
   - `id_grupo` (PK), `id_asignatura` (FK), `id_periodo` (FK), `id_docente` (FK), `codigo_grupo` (ej. "G01"), `cupo_maximo`, `cupo_ocupado`.
10. **Horario:** Franjas de clase asignadas a un grupo.
    - `id_horario` (PK), `id_grupo` (FK), `dia_semana` (1-7), `hora_inicio`, `hora_fin`, `aula_o_enlace`.
11. **Matricula (Cabecera):** Formalización de la inscripción del estudiante en un periodo académico.
    - `id_matricula` (PK), `id_estudiante` (FK), `id_periodo` (FK), `fecha_registro`, `estado` (Inscrita, Retirada).
12. **DetalleMatricula (Línea de Matrícula):** Asignación específica a un grupo del periodo.
    - `id_detalle` (PK), `id_matricula` (FK), `id_grupo` (FK), `id_asignatura` (FK denormalizada para integridad), `estado` (Matriculado, Retirado).
13. **Calificacion:** Evaluaciones y notas por corte.
    - `id_calificacion` (PK), `id_detalle` (FK), `nombre_corte` (ej. "Parcial 1"), `porcentaje` (ej. 30%), `nota_obtenida`, `fecha_registro`, `id_docente_calificador` (FK).
14. **Auditoria:** Pistas inmutables de operaciones sensibles (WORM).
    - `id_auditoria` (PK, Bigint), `id_usuario` (FK), `accion` (LOGIN, REGISTRO_NOTA, CAMBIO_ROL, MATRICULA), `entidad_afectada`, `id_registro_afectado`, `valores_anteriores` (JSONB), `valores_nuevos` (JSONB), `fecha_hora`, `ip_origen`.

P4-02 · modelo de datos

2. Cardinalidades de las Relaciones

- **Usuario – Rol:** `M:N` (Un usuario puede tener uno o más roles; un rol aplica a múltiples usuarios) vía `UsuarioRol`.
- **Usuario – Estudiante / Docente:** `1:0..1` (Herencia por composición: un usuario puede o no ser estudiante o docente).
- **Programa – Estudiante:** `1:N` (Un programa tiene muchos estudiantes; un estudiante pertenece a un programa principal).
- **Programa – Asignatura:** `1:N` (Un programa estructura muchas asignaturas).
- **Asignatura – Grupo:** `1:N` (Una asignatura se oferta en uno o varios grupos por periodo).
- **PeriodoAcademico – Grupo:** `1:N` (Un periodo agrupa múltiples cursos abiertos).
- **Docente – Grupo:** `1:N` (Un docente dicta cero o varios grupos; un grupo tiene un docente responsable).
- **Grupo – Horario:** `1:N` (Un grupo tiene varias sesiones semanales de clase).
- **Estudiante – Matricula:** `1:N` (Un estudiante se matrícula a lo largo de varios periodos).
- **PeriodoAcademico – Matricula:** `1:N` (Un periodo contiene muchas matrículas).
- **Matricula – DetalleMatricula:** `1:N` (Una matrícula inscribe varias asignaturas/grupos).
- **Grupo – DetalleMatricula:** `1:N` (Un grupo tiene muchos estudiantes inscritos).
- **DetalleMatricula – Calificacion:** `1:N` (Una asignatura matriculada acumula varios cortes de nota).
- **Usuario – Auditoria:** `1:N` (Un usuario genera múltiples eventos de trazabilidad).

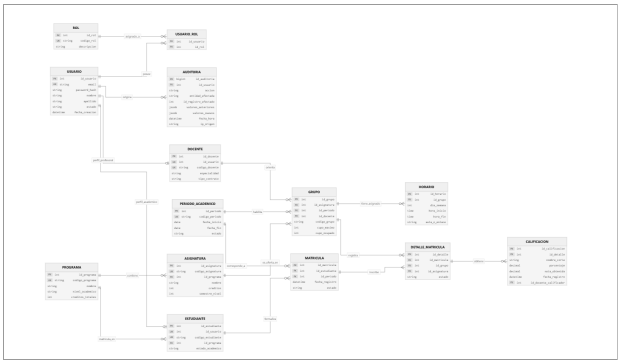
P4-03 · modelo de datos

3. Restricciones Técnicas para Evitar Matrículas Duplicadas y Sobrecupo

Para asegurar el **RNF-07 (Integridad y consistencia)**, el sistema no se confía únicamente de la validación en código Back-End; impone **restricciones formales a nivel de Base de Datos**:

1. **Unicidad de Matrícula por Periodo (Nivel Cabecera):**
   - Restricción: `UNIQUE (id_estudiante, id_periodo)` en la tabla `Matricula`.
   - *Efecto:* Impide que un estudiante abra más de un proceso de matrícula para el mismo periodo académico.
2. **Prevención de Asignatura Duplicada en Diferentes Grupos (Nivel Detalle):**
   - Restricción: `UNIQUE (id_matricula, id_asignatura)` en la tabla `DetalleMatricula`.
   - *Efecto:* Aunque un estudiante intente registrar el Grupo 1 y el Grupo 2 de la misma asignatura (ej. "Cálculo I"), la base de datos abortará la transacción por colisión de clave única.
3. **Prevención de Inscripción Duplicada en el Mismo Grupo:**
   - Restricción: `UNIQUE (id_matricula, id_grupo)` en `DetalleMatricula`.
   - *Efecto:* Evita inserciones redundantes concurrentes en el mismo grupo.
4. **Control Estricto de Cupo Máximo (Anti-Sobrecupo / Overbooking):**
   - Restricción de verificación en `Grupo`: `CHECK (cupo_ocupado <= cupo_maximo)`.
   - *Mecanismo Transaccional:* En el momento de la inscripción, el Back-End ejecuta un bloqueo de fila (`SELECT cupo_ocupado, cupo_maximo FROM Grupo WHERE id_grupo = :id FOR UPDATE`), valida disponibilidad, incrementa `cupo_ocupado = cupo_ocupado + 1` e inserta el `DetalleMatricula` bajo la misma transacción ACID atómica.
5. **Prevención de Cruce de Horarios (Solapamiento Temporal):**
   - Validación a nivel de servicio y soportada opcionalmente mediante restricciones de exclusión temporal en PostgreSQL (`EXCLUDE USING GIST` sobre rangos de tiempo y días), validando que no coincidan dos grupos con `(dia_semana, hora_inicio, hora_fin)` para el mismo estudiante.

P4-04 · modelo de datos



P4-05 · modelo de datos

Modela el diagrama de secuencia del proceso de matrícula de un estudiante en un grupo.
Incluye los participantes (estudiante o coordinador, interfaz web, API, módulos internos, base de datos y auditoría), el orden de los mensajes, la validación de cupo, la validación de matrícula duplicada y qué pasa si dos solicitudes llegan al mismo tiempo por el último cupo.
Genera el diagrama en sintaxis Mermaid (sequenceDiagram)

P5-01 · secuencia de la matrícula

A continuación, se detalla el diseño de interacción y concurrencia para el proceso de matrícula de una asignatura/grupo.

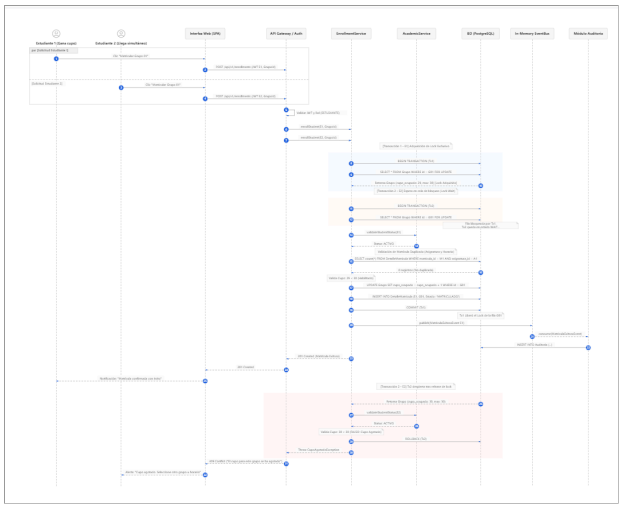
Para resolver de forma estricta el **RNF-07 (Integridad y consistencia)** y evitar sobrecupos (*overbooking*), la arquitectura implementa un mecanismo de **bloqueo pesimista a nivel de fila (`SELECT ... FOR UPDATE`)** dentro de una transacción ACID en PostgreSQL.

---

1. Descripción del Flujo y Resolución de Concurrencia

1. **Autenticación y Autorización:** La API intercepta las solicitudes, valida la firma del token JWT y comprueba que los usuarios posean el rol `ESTUDIANTE` (o `COORDINADOR`) habilitado.
2. **Inicio de Transacción Atómica:** Se abre una transacción en la base de datos para cada solicitud.
3. **Escenario de Concurrencia Crítica (Último Cupo):**
   - Las solicitudes de **Estudiante 1** y **Estudiante 2** llegan prácticamente al mismo milisegundo por el cupo 30 de 30 (`cupo_ocupado = 29`, `cupo_maximo = 30`).
   - La **Transacción 1 (Tx1)** adquiere primero el cerrojo exclusivo sobre el registro del grupo mediante `SELECT ... FOR UPDATE`.
   - La **Transacción 2 (Tx2)** entra en **estado de espera bloqueada (*WAIT*)** en el motor de base de datos hasta que Tx1 libere el cerrojo.
4. **Validaciones en Tx1:**
   - **Duplicidad:** Verifica que el Estudiante 1 no tenga ya matriculada esa asignatura en el periodo actual ni cruce de horario.
   - **Cupo:** Evalúa `cupo_ocupado (29) < cupo_maximo (30)`. Es válido.
   - **Persistencia:** Incrementa `cupo_ocupado` a 30, inserta el registro en `DetalleMatricula` y ejecuta `COMMIT`.
   - **Auditoría Asíncrona:** Emite el evento `MatriculaExitosaEvent` hacia el bus de eventos en memoria sin frenar la respuesta al usuario.
5. **Resolución de Tx2 (Solicitud Rechazada):**
   - Tras el `COMMIT` de Tx1, se libera el cerrojo y **Tx2 despierta**, leyendo inmediatamente el estado actualizado: `cupo_ocupado = 30`.
   - Tx2 evalúa la condición de cupo: `30 < 30` resulta **falso**.
   - Tx2 ejecuta `ROLLBACK` inmediato y el servicio arroja la excepción de dominio `CupoAgotadoException`.
   - La API responde al Estudiante 2 un código HTTP `409 Conflict`, garantizando **cero sobrecupo**.

P5-02 · secuencia de la matrícula



P5-03 · secuencia de la matrícula

3. Consideraciones de Resiliencia y Rendimiento en el Flujo

1. **Tiempo de Retención de Bloqueos (*Lock Contention*):**
   - La transacción solo mantiene el bloqueo de fila durante las micro-operaciones de lectura y escritura (un tiempo típico de ≤ 15 ms). Esto garantiza que, incluso con múltiples estudiantes compitiendo simultáneamente, la cola de espera de la base de datos se desahogue rápidamente sin provocar agotamiento de conexiones en el pool de la aplicación.
2. **Prevención de *Deadlocks* (Bloqueos Mutuos):**
   - Cuando un estudiante inscribe varias asignaturas en un mismo envío (*batch enrollment*), el backend ordena previamente los `id_grupo` de forma ascendente antes de solicitar los bloqueos `FOR UPDATE`. Esto previene de forma determinista cualquier situación de abrazo mortal entre transacciones concurrentes.
3. **Desacoplamiento de la Auditoría:**
   - La escritura en la tabla `Auditoria` ocurre de manera asíncrona mediante el `EventBus`. Así, la persistencia del log de auditoría no retrasa la confirmación de la matrícula al estudiante ni retiene recursos de la transacción crítica de cupos.

User • 1:00 p.m.

Propón el modelo de despliegue del SGA: nodos o servicios donde se ejecutan la interfaz web, la API, la base de datos y los servicios complementarios.
Indica los canales de comunicación, los controles de seguridad (HTTPS, firewall, manejo de contraseñas, tokens), cómo se logra la disponibilidad del 99,5 % y cómo serían las copias de seguridad y la restauración.
Considera que es una institución con presupuesto moderado. Genera el diagrama en sintaxis Mermaid.

P5-04 · secuencia de la matrícula

Para una institución de educación superior con **presupuesto moderado**, la arquitectura de infraestructura debe evitar la complejidad y el alto costo de plataformas como Kubernetes multi-región, priorizando una estrategia **pragmática, altamente automatizada y de costo eficiente** basada en servicios administrados estándar o máquinas virtuales optimizadas con contenedores Docker.

1. Nodos y Servicios del Despliegue

Capa / Servicio	Componente Tecnológico	Propósito y Dimensionamiento
Edge / CDN	Cloudflare (Plan Free/Pro)	Terminación TLS, mitigación de DDoS, WAF básico y distribución de contenido estático (SPA). Costo casi nulo.
Frontend Web	Almacenamiento de Objetos (S3 / MinIO) + CDN	Aloja la interfaz web compilada (HTML5/JS/CSS). Elimina el consumo de CPU en servidores para servir archivos estáticos.
Balanceador de Carga	Nginx / HAProxy o Cloud Load Balancer	Proxy inverso que distribuye tráfico HTTPS hacia los nodos del backend y monitorea *health checks*.
Backend API (Monolito)	2 Instancias de Cómputo (VM o Contenedores)	Ejecutan el artefacto monolítico modular (Node.js/NestJS, Spring Boot o .NET 8). 2 vCPU y 4 GB RAM por nodo bastan para 300 concurrentes.
Caché y Mensajería	Redis (Instancia Standalone / Docker)	Almacena listas negras de tokens JWT, control de tasa (*rate limiting*) y caché de catálogos académicos de solo lectura.
Base de Datos	PostgreSQL Administrado (Primary + Standby)	Motor relacional transaccional (ej. AWS RDS db.t4g.medium o DigitalOcean Managed DB) con réplica de respaldo para conmutación por error.
Almacenamiento Backups	Almacenamiento en Nube Fría (S3 / Wasabi / B2)	Destino externo para copias de seguridad encriptadas fuera del centro de datos principal (Regla 3-2-1).

P6-01 · despliegue

2. Canales de Comunicación y Controles de Seguridad

- **Canales y Protocolos:**
  - **Cliente a Edge / Balanceador:** Exclusivamente `HTTPS (TLS 1.3)` sobre el puerto `443`. Se fuerza redirección automática de HTTP (80) a HTTPS.
  - **Balanceador a Nodos de Backend:** Red Privada Virtual (VPC) aislada mediante `HTTP/2` o `TLS interno`.
  - **Backend a Base de Datos y Caché:** Conexiones cifradas en tránsito vía `PostgreSQL Wire Protocol con SSL (verify-full)` en puerto 5432 y `Redis RESP con autenticación de contraseña (AUTH)` en puerto 6379, restringidas a la subred privada.
- **Segmentación de Red y Firewall:**
  - La base de datos y la caché residen en una **Subred Privada** sin dirección IP pública.
  - Se configuran Grupos de Seguridad / Firewall (UFW/AWS Security Groups) para que el puerto de la base de datos (5432) solo acepte tráfico proveniente de las IPs internas de los nodos de la API.
- **Manejo de Contraseñas y Secretos:**
  - **Contraseñas de Usuarios:** Cifrado unidireccional irreversible mediante **Argon2id** (o **BCrypt** con factor de costo 12). Nunca se persisten ni viajan en texto plano.
  - **Secretos del Sistema:** Las llaves criptográficas JWT, credenciales de BD y conexiones externas no se alojan en el repositorio de código; se inyectan en tiempo de arranque mediante variables de entorno seguras gestionadas por un servicio centralizado (ej. AWS Secrets Manager, Doppler o Vault).
- **Gestión de Sesiones y Tokens (JWT):**
  - **Access Token:** Formato JWT de vida muy corta (15 minutos) almacenado solo en memoria del navegador del cliente (SPA).
  - **Refresh Token:** De vida más prolongada (7 días), almacenado en una cookie con atributos `HttpOnly`, `Secure` y `SameSite=Strict` para prevenir ataques de Cross-Site Scripting (XSS) y CSRF. Se aplica rotación de refresh tokens en cada renovación.

P6-02 · despliegue

3. Estrategia de Disponibilidad (Objetivo 99,5 %)

Un SLA del 99,5 % mensual permite un **margen máximo de indisponibilidad no planificada de** $\approx 3,65$ **horas al mes**. Se garantiza mediante:

1. **Redundancia Activo-Activo en Capa de Aplicación:**
   - Se despliegan dos instancias idénticas del backend detrás del balanceador de carga. Si una instancia falla o consume el 100% de memoria, el balanceador redirige el 100% del tráfico al nodo sobreviviente en menos de 5 segundos.
2. **Despliegues sin Interrupción (*Zero-Downtime Deployments*):**
   - Se utiliza la estrategia **Rolling Update** (o Blue/Green): se actualiza la Instancia 1 mientras la Instancia 2 atiende tráfico; una vez superado el *health check*, se conmuta y se actualiza la Instancia 2. Esto garantiza que las actualizaciones regulares no consuman el presupuesto de caída.
3. **Alta Disponibilidad en Base de Datos:**
   - Uso de un PostgreSQL con réplica de reserva sincrónica o semi-sincrónica (*Standby Hot-Replica*). En caso de fallo crítico en el nodo primario, el proveedor o el gestor de clúster promueve la réplica a primaria automáticamente (*Failover automático* en 60 a 120 segundos).

P6-03 · despliegue

4. Respaldo y Recuperación ante Desastres (Disaster Recovery)

Para ajustarse al **RNF-09** con métricas institucionales razonables:

- **RPO (Punto de Recuperación Objetivo):** ≤ 15 minutos (máxima pérdida tolerable de datos).
- **RTO (Tiempo de Recuperación Objetivo):** ≤ 2 horas (tiempo máximo para volver a estar operativos tras catástrofe).

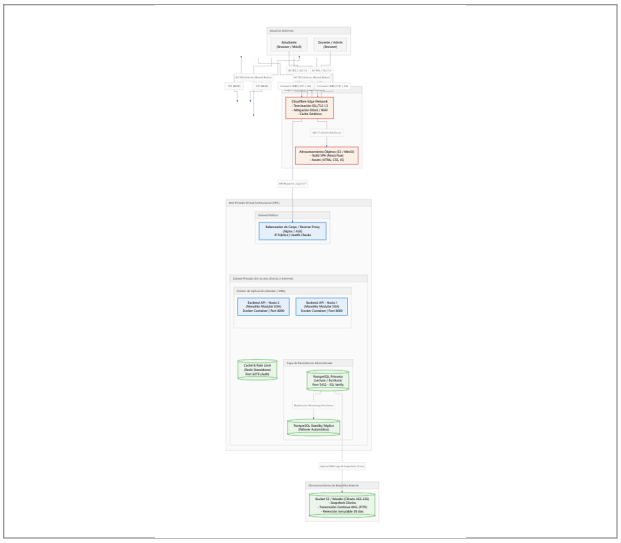
Procedimiento de Copias de Seguridad:

1. **Copias Completas Diarias:** A las 01:00 AM se ejecuta un volcado completo (`pg_dump` consistente o snapshot de volumen de BD) comprimido y cifrado con clave simétrica AES-256.
2. **Archivado Continuo de WAL (*Write-Ahead Logging*):** Los registros binarios de transacciones (WAL) se transmiten continuamente cada 10 minutos hacia un bucket de almacenamiento de objetos externo (S3/Wasabi). Esto permite **Recuperación en el Punto en el Tiempo (*Point-In-Time Recovery - PITR*)**.
3. **Inmutabilidad y Retención:** Política de retención de 30 días con política *Object Lock* (WORM) para evitar borrado accidental o por ransomware.

Procedimiento de Restauración:

- Script automatizado (`restore_db.sh`) probado trimestralmente en un entorno de pruebas (*staging*). Descarga el último snapshot diario, lo aplica sobre una nueva instancia de PostgreSQL y reaplica los archivos WAL hasta el minuto previo al fallo.

P6-04 · despliegue



P6-05 · despliegue

Ahora actúa como evaluador externo y crítica la arquitectura que propusiste en esta conversación.

1. ¿Qué requerimientos funcionales o no funcionales quedaron sin cubrir o cubiertos de forma débil?
2. ¿Qué puntos únicos de falla existen?
3. ¿Qué riesgos de seguridad y privacidad ves?
4. ¿Cuáles son los límites de escalabilidad?
5. ¿Qué decisiones requieren validación adicional y cómo se validarían (pruebas de carga, pruebas de concurrencia, simulacros de restauración)? Sé concreto y no repitas lo que ya dijiste

P7-01 · autocrítica

Como evaluador externo independiente, presento la auditoría técnica y crítica constructiva a la arquitectura propuesta:

1. Requerimientos Sin Cubrir o Cubiertos Débilmente

- **RF-05 (Prerrequisitos y Correquisitos Académicos):** El modelo de datos y el flujo de matrícula ignoraron por completo las dependencias entre asignaturas (árbol de prelaciones). No hay entidades ni validaciones que impidan a un estudiante inscribir "Cálculo II" sin haber aprobado "Cálculo I".
- **RNF-06 (Accesibilidad WCAG 2.1 AA):** Se mencionó como directriz de diseño, pero carece de soporte arquitectónico: no se definieron herramientas de verificación continua en el pipeline de CI/CD (ej. pruebas automatizadas con *Axe-core* o *Lighthouse CI*), dejando el cumplimiento a la disciplina del desarrollador de Front-End.
- **RNF-10 (Trazabilidad de Requerimientos a Código):** Quedó como una declaración de intenciones. No se especificó un estándar de *Architecture Decision Records* (ADR), versionado semántico de contratos OpenAPI ni etiquetado de commits vinculado a historias/tickets.
- **RF-06 (Gestión de Infraestructura de Aulas vs. Cupos):** El modelo asume que el cupo es solo una propiedad del `Grupo`, ignorando la capacidad física de la entidad `Aula`. Un grupo podría tener configurado un cupo de 40 estudiantes asignado a un aula física con capacidad máxima para 25.
- **RF-10 (Generación Pesada de Reportes):** El módulo de reportes lee directamente del motor de persistencia principal. En fechas críticas de cierre de actas, la generación concurrente de reportes pesados en PDF/Excel degradará el pool de conexiones transaccionales.

P7-02 · autocrítica

2. Puntos Únicos de Falla (SPOFs)

1. **Balanceador de Carga Único en DMZ:** Aunque se propusieron dos nodos de API, solo se ilustró un único nodo de Nginx/HAProxy. Si este balanceador sufre un fallo de proceso, agotamiento de *file descriptors* o kernel panic, todo el sistema queda inaccesible (violando el RNF-03 de 99,5 %).
2. **Instancia Única de Redis (Standalone):** Si el contenedor de Redis falla:
   - El control de tasa (*rate limiting*) queda inoperativo.
   - La lista negra de JWTs se pierde en memoria, dejando vulnerables los tokens revocados.
   - Si la API depende de Redis para cachear catálogos, se producirá un efecto avalancha (*thundering herd*) contra PostgreSQL.
3. **Pérdida de Auditoría por *EventBus* en Memoria:** Si el nodo de la API sufre una caída de tensión o un error de memoria (*OOM Kill*) inmediatamente después de confirmar la transacción en la base de datos pero antes de que el *EventBus* entregue el evento a la cola asíncrona, **la pista de auditoría de esa matrícula se pierde definitivamente**, violando el RF-12.

P7-03 · autocrítica

3. Riesgos de Seguridad y Privacidad

- **Vulnerabilidad del Log de Auditoría ante el Administrador de BD (DBA):** La tabla `audit.logs` reside en la misma base de datos relacional. Un DBA con privilegios de superusuario o un atacante que logre una inyección SQL a nivel administrativo puede ejecutar `UPDATE` o `DELETE` sobre las pistas de auditoría para encubrir alteraciones de calificaciones. **No es un almacenamiento WORM criptográficamente demostrable**.
- **Ambigüedad en Usuarios con Roles Múltiples:** En el modelo, `UsuarioRol` es M:N. Si un usuario es simultáneamente `Docente` de un departamento y `Estudiante` de posgrado, el diseño no contempla un mecanismo de **Contexto de Sesión Activo**. Un estudiante-docente podría interactuar con endpoints de calificaciones sin una validación estricta del perfil con el que inició la sesión actual.
- **Falta de Anonimización en Calificaciones (RNF-11):** Los endpoints exponen directamente la relación `EstudianteId` ↔ `Nota`. No se contemplaron esquemas de calificación a ciegas (*blind grading*), lo que expone a la institución a riesgos de sesgo o filtración de datos sensibles de rendimiento académico.

P7-04 · autocrítica

4. Límites de Escalabilidad

- **Contención Extrema en el Cerrojo de Fila (`SELECT ... FOR UPDATE`):**
  - *El cuello de botella:* Si se abren las matrículas a las 08:00 AM y 250 de los 300 usuarios intentan matricularse simultáneamente en la misma asignatura popular, las 250 transacciones se serializan una a una sobre el mismo registro de la tabla `Grupo`.
  - *Consecuencia:* Aunque la latencia de una transacción individual sea de 15 ms, una cola de 200 transacciones esperando por el mismo lock acumulará demoras de más de 3 segundos, disparando *timeouts* y superando el límite del **RNF-02**.
- **Agotamiento del Pool de Conexiones de Base de Datos:** PostgreSQL utiliza un modelo de un proceso por conexión (`fork`). Dos instancias de API abriendo conexiones concurrentes contra un motor moderado pueden agotar rápidamente el parámetro `max_connections` (típicamente 100-200 conexiones en servidores modestos) si no se interpone un *connection pooler* transaccional como **PgBouncer**.
- **Techo Vertical del Monolito:** Toda la carga (escrituras críticas de matrícula, subida de notas, analítica pesada y autenticación) compite por la CPU y el I/O del mismo motor de base de datos primario.

P7-05 · autocrítica

5. Decisiones que Requieren Validación Adicional y Planes de Prueba

Decisión Arquitectónica Crítica	Riesgo Asociado	Plan de Validación Propuesto
Bloqueo Pesimista en Cupos (`FOR UPDATE`)	*Deadlocks*, colas de espera excesivas y agotamiento de conexiones en picos de alta demanda.	**Prueba de Concurrencia Extrema:** Ejecutar una prueba con herramientas como *k6* o *Gatling* simulando a 300 usuarios disparando la matrícula sobre un único grupo con 5 cupos restantes en una ventana de 2 segundos. Medir contención, latencia en P95/P99 y verificar matemáticamente cero sobrecupos.
Resiliencia de Auditoría vía EventBus	Pérdida de eventos de auditoría por caídas abruptas del proceso.	**Prueba de Caos / Falla Inyectada:** Enviar un flujo continuo de matrículas y matar el proceso del contenedor (`kill -9`) en momentos aleatorios. Comparar registros de `DetalleMatricula` contra registros en `audit.logs`. Si hay discrepancias, rediseñar usando el **Patrón Outbox Transaccional**.
RPO ≤ 15 min y RTO ≤ 2 h	Estimaciones teóricas no comprobadas sobre restauración de datos.	**Simulacro de Desastre (Fire Drill):** Corromper intencionalmente la base de datos en un entorno réplica, activar cronómetro y ejecutar el procedimiento de restauración desde el bucket externo aplicando Snapshots + WAL streaming hasta el último minuto.
Dimensionamiento para 300 Concurrentes	Sobredimensionamiento o subdimensionamiento de recursos de cómputo en servidores moderados.	**Prueba de Carga y Capacidad (Stress Testing):** Modelar el comportamiento típico (70% lecturas de horarios/notas, 20% matrículas, 10% reportes) y elevar paulatinamente la concurrencia de 50 a 500 usuarios. Identificar el punto de degradación donde el P95 supere los 3 segundos.

P7-06 · autocrítica

Anexo C. Texto de los prompts

P1 · análisis del problema

```
Actúa como arquitecto de software senior. Te voy a dar el caso de un Sistema de Gestión Académica (SGA) para una institución de
educación superior. Necesito analizarlo antes de diseñar la arquitectura.

CONTEXTO: La institución requiere un SGA que centralice la información de estudiantes, docentes, programas académicos, asignaturas,
grupos, matrículas, calificaciones y horarios. Hoy parte de la información se maneja en archivos y aplicaciones independientes, lo que
genera duplicidad de datos, demoras en la consulta y dificultades de trazabilidad. La solución debe ser segura, mantenible, escalable y
accesible desde navegadores web.

REQUERIMIENTOS FUNCIONALES:
RF-01 Autenticación: iniciar y cerrar sesión con credenciales individuales.
RF-02 Roles y permisos: mínimo administrador, coordinador académico, docente y estudiante; restringir funciones según permisos.
RF-03 Gestión de estudiantes: registrar, consultar, actualizar y desactivar estudiantes.
RF-04 Gestión de docentes: registrar, consultar, actualizar y desactivar docentes.
RF-05 Programas y asignaturas: crear y actualizar programas, planes de estudio y asignaturas (código, nombre, créditos, semestre o
nivel).
RF-06 Grupos y periodos: configurar periodos académicos, grupos, cupos, docentes asignados, horarios y aulas o enlaces.
RF-07 Matrícula: matricular estudiantes en asignaturas o grupos de un periodo, validar cupos y evitar matrículas duplicadas.
RF-08 Calificaciones: el docente registra y actualiza calificaciones de los estudiantes de sus grupos.
RF-09 Consulta de resultados: el estudiante consulta asignaturas matriculadas, horarios y calificaciones publicadas.
RF-10 Reportes: reportes básicos de matriculados, grupos, asignaturas y resultados, con filtros por periodo y programa.
RF-11 Búsqueda y filtros: buscar y filtrar por identificadores, nombres, programa, periodo o estado.
RF-12 Auditoría: registrar inicios de sesión, cambios de roles, matrículas y modificaciones de calificaciones (usuario, fecha, acción,
registro afectado).

REQUERIMIENTOS NO FUNCIONALES:
RNF-01 Seguridad: HTTPS, contraseñas no en texto plano, acceso por roles y permisos.
RNF-02 Rendimiento: el 95 % de las consultas comunes responde en máximo 3 segundos con hasta 300 usuarios concurrentes.
RNF-03 Disponibilidad: objetivo mensual de al menos 99,5 %, excluyendo mantenimientos programados.
RNF-04 Escalabilidad: aumentar recursos y atender crecimiento sin rediseñar todo el sistema.
RNF-05 Mantenibilidad: separar responsabilidades, bajo acoplamiento y cohesión adecuada.
RNF-06 Usabilidad y accesibilidad: interfaz consistente, adaptable a móvil, etiquetas claras, contraste y operación por teclado.
RNF-07 Integridad y consistencia: matrícula y calificaciones validan reglas de negocio y preservan consistencia ante errores o
solicitudes simultáneas.
RNF-08 Interoperabilidad: interfaces documentadas, preferiblemente API REST.
RNF-09 Respaldo y recuperación: copias periódicas y procedimiento de restauración.
RNF-10 Trazabilidad: relacionar decisiones y requerimientos con componentes, modelos y evidencias.
RNF-11 Privacidad: limitar acceso a datos personales y académicos según el rol.
RNF-12 Compatibilidad: navegadores recientes y distintos tamaños de pantalla.

TAREA:
1. Identifica los actores principales y secundarios.
2. Propón supuestos razonables donde el caso no es explícito.
3. Lista las restricciones técnicas y del negocio.
4. Extrae las reglas de negocio implícitas en los requerimientos (por ejemplo, sobre cupos, matrículas duplicadas y quién puede
modificar calificaciones).
5. Clasifica cada RNF según el atributo de calidad que representa.
Responde en español, con tablas cuando ayuden.
```

P2 · propuesta de estilo arquitectónico

```
Con base en el análisis anterior, propón el estilo arquitectónico para el SGA.
Compara al menos tres opciones: arquitectura en capas, monolito modular y microservicios.
Para cada una evalúa: complejidad de desarrollo y operación, rendimiento con 300 usuarios concurrentes, disponibilidad del 99,5 %,
escalabilidad, mantenibilidad y consistencia en la matrícula ante solicitudes simultáneas.
Al final recomienda una opción y justifícala frente a los requerimientos no funcionales. Responde en español con una tabla comparativa.
```

P3 · componentes y vista lógica

```
Para la arquitectura que recomendaste, describe la vista lógica:
los módulos o componentes principales, la responsabilidad de cada uno, las interfaces que exponen (por ejemplo, endpoints REST
principales) y las dependencias entre ellos.
Debe quedar claro cómo se organizan autenticación y autorización, gestión académica, matrículas, calificaciones, reportes y auditoría.
Genera además el diagrama en sintaxis Mermaid.
```

P4 · modelo de datos conceptual

```
Propón el modelo de datos conceptual del SGA con las entidades: Usuario, Rol, Estudiante, Docente, Programa, Asignatura,
PeriodoAcadémico, Grupo, Matrícula y Calificación, más las que consideres necesarias (por ejemplo, para horarios o auditoría).
Indica los atributos principales y las cardinalidades de cada relación, y explica las restricciones que evitan matrículas duplicadas.
Genera el diagrama entidad-relación en sintaxis Mermaid (erDiagram).
```

P5 · secuencia de la matrícula

```
Modela el diagrama de secuencia del proceso de matrícula de un estudiante en un grupo.
Incluye los participantes (estudiante o coordinador, interfaz web, API, módulos internos, base de datos y auditoría), el orden de los mensajes, la validación de cupo, la validación de matrícula duplicada y qué pasa si dos solicitudes llegan al mismo tiempo por el último cupo.
Genera el diagrama en sintaxis Mermaid (sequenceDiagram).
```

P6 · despliegue, seguridad y respaldo

```
Propón el modelo de despliegue del SGA: nodos o servicios donde se ejecutan la interfaz web, la API, la base de datos y los servicios complementarios.
Indica los canales de comunicación, los controles de seguridad (HTTPS, firewall, manejo de contraseñas, tokens), cómo se logra la disponibilidad del 99,5 % y cómo serían las copias de seguridad y la restauración.
Considera que es una institución con presupuesto moderado. Genera el diagrama en sintaxis Mermaid.
```

P7 · revisión crítica de la propia propuesta

```
Ahora actúa como evaluador externo y critica la arquitectura que propusiste en esta conversación.
1. ¿Qué requerimientos funcionales o no funcionales quedaron sin cubrir o cubiertos de forma débil?
2. ¿Qué puntos únicos de falla existen?
3. ¿Qué riesgos de seguridad y privacidad ves?
4. ¿Cuáles son los límites de escalabilidad?
5. ¿Qué decisiones requieren validación adicional y cómo se validarían (pruebas de carga, pruebas de concurrencia, simulacros de restauración)?
Sé concreto y no repitas lo que ya dijiste.
```